



UNIVERSITÉ DE MONTPELLIER

Chasse aux triangles dans Jeux De Mots

Réalisé par

- Groupe SHLK -

Sofia ABDON

Hajar BOUMEZGANE

Khaled EL BAF

Lynna MOUSSAOUI

Dirigé par

Mathieu LAFOURCADE

Rapporteur

SERIAL Abdelhak-Djamel

Projet réalisé dans le cadre du

TER

M1 – Informatique Ico

Année scolaire 2024 – 2025

REMERCIEMENT

Nous tenons à exprimer notre profonde gratitude envers toutes les personnes qui ont contribué à la réalisation de ce projet de TER.

Tout d'abord, nous adressons nos sincères remerciements à notre encadrant, **M. Mathieu LAFOURCADE**, pour son accompagnement précieux, ses conseils avisés et sa disponibilité tout au long de ce travail. Son expertise et son soutien ont été déterminants pour la réussite de ce projet.

Nous remercions également notre rapporteur, **M. SERIAI Abdelhak-Djamel**, pour le temps consacré à l'évaluation de notre travail.

Nos remerciements s'étendent à l'ensemble des enseignants du Master 1 Informatique, parcours ICO, pour leurs enseignements enrichissants et leur engagement tout au long de cette année. Leurs cours ont grandement contribué à notre formation et à l'aboutissement de ce projet.

Enfin, nous souhaitons remercier nos proches pour leur soutien indéfectible et leurs encouragements durant cette année exigeante mais passionnante.

GLOSSAIRE

Traitement Automatique du Langage (TAL) : Domaine pluridisciplinaire impliquant la linguistique, l'informatique et l'intelligence artificielle qui vise à créer des outils de traitement de texte ou de la parole (même signée) pour diverses applications.

Schéma d'inférence : Modèle logique décrivant comment certaines relations entre des concepts permettent d'en déduire d'autres.

API (Application Programming Interface) : Interface logicielle qui permet de « connecter » un logiciel ou un service à un autre logiciel ou service afin d'échanger des données et des fonctionnalités.

Endpoint : l'endroit où une API reçoit des requêtes. Pour la plupart des services, ces endpoints sont des URL, tout comme ceux que vous utilisez pour naviguer sur un site web. Il peut être visualisé comme une adresse spécifique sur un serveur, généralement sous forme d'URL, où les requêtes API sont envoyées et d'où les réponses sont reçues. Chaque endpoint est associé à une fonction spécifique, rendant la modularité et la flexibilité des applications possibles.

RELATIONS

Dans le cadre de cette étude, nous présentons tout d'abord un tableau récapitulatif des relations sémantiques les plus pertinentes que nous avons identifiées. Ce tableau comprend pour chaque relation son identifiant numériques et sa signification :

N°	Nom	Signification
5	r_syn	Il est demandé d'énumérer les synonymes ou quasi-synonymes de ce terme. - [conversif : r_syn] - $[1 \leq q=1 \leq 5]$
6	r_isa	Il est demandé d'énumérer les GENERIQUES/hyperonymes du terme. Par exemple, 'animal' et 'mammifère' sont des génériques de 'chat'. - [conversif : r_hypo] - $[1 \leq q=1 \leq 2]$
8	r_hypo	Il est demandé d'énumérer des SPECIFIQUES/hyponymes du terme. Par exemple, 'mouche', 'abeille', 'guêpe' pour 'insecte'. - [conversif : r_isa] - $[1 \leq q=1 \leq 2]$
9	r_has_part	Il faut donner des PARTIES/constituants/éléments (a pour méronymes) du mot cible. Par exemple, 'voiture' a comme parties : 'porte', 'roue', 'moteur', ... - [conversif : r_holo] - $[1 \leq q=1 \leq 2]$
10	r_holo	Il est demandé d'énumérer des 'TOUT' (a pour holonymes) de l'objet en question. Pour 'main', on aura 'bras', 'corps', 'personne', etc... Le tout est aussi l'ensemble comme 'classe' pour 'élève'. - [conversif : r_has_part] - $[1 \leq q=1 \leq 2]$
11	r_locution	A partir d'un terme, il est demandé d'énumérer les locutions, expression ou mots composés en rapport avec ce terme. Par exemple, pour 'moulin', on pourra avoir 'moulin à vent', 'moulin à eau', 'moulin à café'. Pour 'vendre', on pourra avoir 'vendre la peau de l'ours avant de l'avoir tué', 'vendre à perte', etc.. - $[1 \leq q=1 \leq 3]$
13	r_agent	L'agent (qu'on appelle aussi le sujet) est l'entité qui effectue l'action, OU la subit pour des formes passives ou des verbes d'état. Par exemple, dans - Le chat mange la souris -, l'agent est le chat. Des agents typiques de 'courir' peuvent être 'sportif', 'enfant',... (manger r_agent chat) - [conversif : r_agent-1] - $[2 \leq q=4.73 \leq 10]$
17	r_carac	Pour un terme donné, souvent un objet, il est demandé d'en énumérer les caractéristiques (adjectifs) possibles/typiques. Par exemple, 'liquide', 'froide', 'chaude', pour 'eau'. - [conversif : r_carac-1] - $[1 \leq q=3.33 \leq 5]$
19	r_lemma	Le lemme (par exemple 'mange' a pour lemme 'manger' ; 'avons' a pour lemme 'avoir' ou 'avoir'). - $[1 \leq q=1 \leq 2]$
23	carac_1	Quels sont les objets (des noms) possédant typiquement/possiblement la caractéristique suivante ? Par exemple, 'soleil', 'feu', pour 'chaud'. - [conversif : r_carac] - $[1 \leq q=6.06 \leq 10]$
24	r_agent_1	Que peut faire ce SUJET ? (par exemple chat => miauler, griffer, etc.) (chat r_agent-1 manger) - [conversif : r_agent] - $[1 \leq q=6.41 \leq 10]$

26	r_patient-1	(inverse de r_patient) Que peut-on faire à cet OBJET. Pour 'pomme', on pourrait avoir 'manger', 'croquer', couper', 'épucher', etc. (pomme r_patient-1 manger) - [conversif : r_patient] - [1 ≤ q=1 ≤ 10]
27	r_domain-1	inverse de r_domain : à un domaine, on associe des termes - [conversif : r_domain] - [1 ≤ q=1 ≤ 1]
28	r_lieu_1	L'instrument est l'objet avec lequel on fait l'action. Dans - Il mange sa salade avec une fourchette -, fourchette est l'instrument. On demande ici, ce qu'on peut faire avec un instrument donné... (scie r_instr-1 scier) - [conversif : r_instr] - [1 ≤ q=1 ≤ 10]
30	r_lieu_action	A partir d'un lieu, énumérer les actions typiques possibles dans ce lieu. - [conversif : r_action_lieu] - [1 ≤ q=1 ≤ 1]
32	r_sentiment	Pour un terme donné, évoquer des mots liés à des SENTIMENTS ou des EMOTIONS que vous pourriez associer à ce terme. Par exemple, la joie, le plaisir, le dégoût, la peur, la haine, l'amour, l'indifférence, l'envie, avoir peur, horrible, etc. - [conversif : r_sentiment-1] - [1 ≤ q=7.29 ≤ 10]
35	r_meaning	Quels SENS/SIGNIFICATIONS pouvez vous donner au terme proposé. Il s'agira de termes (des gloses) évoquant chacun des sens possibles, par exemple : 'forces de l'ordre', 'contrat d'assurance', 'police typographique', ... pour 'police'. - [2 ≤ q=2.43 ≤ 4]
36	r_infopot	Information sémantique potentielle - [1 ≤ q=0.85 ≤ 2]
39	r_verbe_action	du verbe vers l'action. Par exemple, construire -> construction , jardiner -> jardinage . C'est un terme directement dérivé (ayant la même racine). Applicable que pour un verbe et inverse de la relation 40 (action vers verbe). - [conversif : r_action-verbe] - [1 ≤ q=1 ≤ 2]
49	r_time	Donner une valeur temporelle -quel moment- peut-on associer au terme indiqué (par exemple 'dormir' -> nuit, 'bronzer' -> été, 'fatigue' -> 'soir') - [1 ≤ q=1.48 ≤ 2]
64	r_has_instance	Une instance d'un 'type' est un individu particulier de ce type. Il s'agit d'une entité nommée (personne, lieu, organisation, etc) - par exemple, 'cheval' a pour instance possible 'Jolly Jumper', ou encore 'transatlantique' a pour instance possible 'Titanic'. - [conversif : r_is_instance_of] - [1 ≤ q=1 ≤ 3]
115	r_sentiment_1	Pour un SENTIMENT ou EMOTION donné, il est demandé d'énumérer les termes que vous pourriez associer. Par exemple, pour 'joie', on aurait 'cadeau', 'naissance', 'bonne nouvelle', etc. - [conversif : r_sentiment] - [5 ≤ q=5 ≤ 10]
139	r_nomprop-adj	Pour un nom de propriété, donner l'adjectif correspondant. Par exemple, pour 'friabilité' -> 'friable' - [conversif : r_adj-nomprop] - [1 ≤ q=1 ≤ 2]
999	r_inhib	Relation d'inhibition, le terme inhibe les termes suivants... ce terme a tendance à exclure le terme associé. - [1 ≤ q=1 ≤ 2]

RESUME

Ce rapport présente une étude approfondie des triangles sémantiques au sein du réseau lexical JeuxDeMots, une ressource majeure pour l'analyse des relations entre concepts du français. L'objectif principal de ce travail est de mettre en lumière les structures récurrentes et les schémas d'inférence qui émergent de la combinaison de différentes relations lexicales. En s'appuyant sur une extraction systématique des triangles, nous avons pu identifier les « chapeaux », c'est-à-dire les paires de relations secondaires qui, associées à une relation principale, forment des motifs structurants au sein du graphe.

L'analyse quantitative et qualitative des triangles détectés révèle une organisation hiérarchique et modulaire du lexique, où certaines combinaisons de relations jouent un rôle central dans la cohérence sémantique du réseau. Les résultats mettent en évidence la fréquence et la diversité des chapeaux, ainsi que l'existence de clusters sémantiques et de chemins d'inférence privilégiés. Cette structuration n'est pas aléatoire : elle reflète des processus cognitifs et linguistiques fondamentaux, tels que la catégorisation, la définition ou l'association d'idées.

Au-delà de l'aspect descriptif, ce travail ouvre des perspectives pour le traitement automatique du langage naturel. Les motifs d'inférence identifiés pourraient enrichir les modèles de représentation du sens, améliorer la désambiguïsation lexicale ou guider la génération automatique de texte. Enfin, la méthodologie développée ici, fondée sur l'analyse de motifs structuraux, est généralisable à d'autres langues ou à d'autres types de réseaux sémantiques, offrant ainsi un cadre d'étude robuste et innovant pour l'exploration des ressources linguistiques à grande échelle.

ABSTRACT

This report presents an in-depth study of semantic triangles within the JeuxDeMots lexical network, a major resource for analyzing relationships between French concepts. The main objective of this work is to highlight the recurring structures and inference patterns that emerge from the combination of different lexical relations. By systematically extracting triangles, we were able to identify "hats," which is, pairs of secondary relations which, together with a primary relation, form structuring motifs within the graph.

The quantitative and qualitative analysis of the detected triangles reveals a hierarchical and modular organization of the lexicon, where certain combinations of relations play a central role in the semantic coherence of the network. The results highlight the frequency and diversity of these hats, as well as the existence of semantic clusters and preferred inference paths. This structuring is not random: it reflects fundamental cognitive and linguistic processes, such as categorization, definition, or association of ideas.

Beyond the descriptive aspect, this work opens up new perspectives for natural language processing. The identified inference patterns could enrich semantic representation models, improve lexical disambiguation, or guide automatic text generation. Finally, the methodology developed here, based on the analysis of structural motifs, is generalizable to other languages or types of semantic networks, thus offering a robust and innovative framework for the exploration of large-scale linguistic resources.

TABLE DES MATIERES

REMERCIEMENT	2
GLOSSAIRE	3
RELATIONS	4
RESUME.....	6
ABSTRACT	7
INTRODUCTION.....	10
PARTIE 1 : ÉTAT DE L'ART.....	12
1. Les réseaux lexico-sémantiques : fondements et évolutions	12
2. Les jeux avec un but (GWAP) : une révolution dans la collecte de données linguistiques....	12
3. L'inférence dans les graphes de connaissances : modèles et algorithmes.....	13
4. La détection de triangles dans les graphes : un outil d'analyse et d'inférence.....	13
5. L'intelligence artificielle explicable (XAI) : vers des systèmes transparents et interprétables	
14	
PARTIE 2 : METHODOLOGIE	15
1. Organisation du projet	15
2. Présentation de l'API JeuxDeMots.....	15
3. Scripts python et algorithmes utilisés	16
3.1. Obstacles rencontrés et solutions	16
3.2. Stratégie de stockage des données.....	18
3.3. Structure et fonctionnement du script	19
4. Utilisation de l'IA dans le projet	22
4.1. Pourquoi utiliser l'IA ?	22
4.2. Les prompts	22
5. Validation et test.....	24
5.1. Les triangles	24
PARTIE 3 : RESULTATS ET EVALUATION	27
1. Statistiques globales	27
1.1. Explications.....	27
1.2. Test effectués	27
1.3. Résultats	28

2. Visualisation des triangles détectés	29
2.1. Explication générale.....	29
2.2. Explications des différents graphiques.....	30
3. Interprétation sémantique	36
3.1. Explications générales	36
3.2. Bases pertinentes	38
3.3. Chapeaux Pertinents.....	39
3.4. Chapeaux non-pertinents	43
3. Discussion sur les résultats	45
4.1. Discussion générale.....	45
4.2. Améliorations	45
CONCLUSION	47
BIBLIOGRAPHIE	49

INTRODUCTION

Dans le cadre de notre Master Informatique à l'Université de Montpellier, nous menons une étude sur JeuxDeMots, un projet innovant qui allie jeu et acquisition de connaissances linguistiques.

Le traitement automatique du langage (TAL) est un domaine en pleine expansion, au cœur duquel se trouvent les graphes sémantiques, outils puissants pour modéliser les relations entre concepts linguistiques. Parmi les initiatives majeures dans ce domaine figure le projet JeuxDeMots, un 'Game With A Purpose' (GWAP) développé au LIRMM en 2007 grâce aux travaux pionniers de notre encadrant, M. Mathieu Lafourcade. Ce jeu sérieux innovant vise à collecter des connaissances lexicales et sémantiques à grande échelle, tout en offrant une expérience ludique aux utilisateurs. Grâce à la participation massive et collaborative des joueurs.

L'objectif principal de JeuxDeMots est de construire et d'enrichir un réseau lexico-sémantique massif pour la langue française. Ce réseau, structuré sous forme de graphe, contient actuellement plus de 20 millions de nœuds représentant des termes linguistiques et environ 850 millions de relations sémantiques entre ces termes. Ces relations, de nature variée (synonymie, causalité, méronymie, etc.), sont validées par le consensus des joueurs, ce qui assure une certaine fiabilité des données tout en permettant une collecte à grande échelle.

Notre étude se concentre sur l'identification et l'analyse des triangles sémantiques au sein de ce graphe. Un triangle sémantique est une structure composée de trois termes reliés entre eux par des relations directes ou indirectes, formant ainsi un cycle logique. Par exemple, si le terme A est lié au terme B, que C est lié à B, et que A est également lié à C, ces trois relations forment un triangle. Ces configurations sont particulièrement intéressantes car elles permettent à la fois de valider la cohérence du graphe et de déduire de nouvelles connaissances par inférence.

L'identification de ces triangles présente plusieurs enjeux majeurs. Tout d'abord, elle permet d'améliorer la qualité du graphe en détectant des relations redondantes. Ensuite, elle offre des possibilités d'inférence sémantique, c'est-à-dire de déduction de relations implicites à partir des relations explicites. Enfin, ces structures peuvent servir de base pour expliquer des relations complexes dans des systèmes d'intelligence artificielle, répondant ainsi au besoin croissant d'IA explicable (XAI).

Pour mener à bien cette étude, nous avons adopté une approche méthodologique rigoureuse. Nous avons d'abord exploré la base de données JeuxDeMots. Ensuite, nous avons conçu et optimisé des requêtes capables d'identifier efficacement les triangles sémantiques dans ce

graphe de grande taille. Enfin, nous avons analysé les résultats pour en extraire des tendances et des motifs récurrents, tout en évaluant la performance de nos requêtes sur un serveur dédié.

Nous commençons par un état de l'art qui situe notre projet dans le paysage scientifique actuel. Nous décrivons ensuite notre méthodologie, en mettant l'accent sur les défis techniques rencontrés et les solutions apportées. Nous présentons ensuite nos résultats, accompagnés d'une analyse critique. Enfin, nous discutons des perspectives ouvertes par cette recherche, notamment en termes d'applications pratiques et d'améliorations futures.

PARTIE 1 : ÉTAT DE L'ART

L'étude des réseaux lexico-sémantiques, et en particulier du projet Jeux De Mots, s'inscrit dans une dynamique de recherche multidisciplinaire, à la croisée de la linguistique computationnelle, de la théorie des graphes, de l'intelligence collective et de l'intelligence artificielle explicable. Cette section propose un panorama des principaux travaux et concepts qui fondent et motivent notre démarche.

1. Les réseaux lexico-sémantiques : fondements et évolutions

Les réseaux lexico-sémantiques sont des bases de connaissances structurées qui modélisent le lexique d'une langue sous forme de graphes : les nœuds représentent des termes ou des concepts, et les arcs, les relations sémantiques qui les unissent (synonymie, antonymie, hyperonymie, méronymie, causalité, etc.). Parmi les pionniers, on trouve WordNet pour l'anglais, qui a servi de modèle à de nombreux projets multilingues comme EuroWordNet ou le Réseau Lexical du Français (RL-fr).

Ces ressources sont devenues des références incontournables pour le traitement automatique du langage (TAL), la traduction automatique, la recherche d'information ou encore la génération de texte. JeuxDeMots se distingue par sa couverture exhaustive du français, la diversité de ses relations et surtout sa méthode de construction participative.

À ce jour, le réseau JeuxDeMots compte plus de 20 millions de nœuds et près de 850 millions de relations, ce qui en fait l'une des ressources les plus riches et dynamiques pour la langue française.

2. Les jeux avec un but (GWAP) : une révolution dans la collecte de données linguistiques

L'émergence des jeux avec un but (GWAP, « Games With A Purpose ») a profondément renouvelé la manière de constituer des bases de données linguistiques. Initiée par Luis von Ahn avec l'ESP Game (pour l'annotation d'images), cette approche exploite l'intelligence collective des joueurs pour résoudre des tâches complexes, traditionnellement réservées à des experts ou à des algorithmes coûteux. Dans le domaine linguistique, JeuxDeMots est l'un des premiers GWAP à viser la construction d'un réseau sémantique.

Les joueurs, en participant à des jeux comme JeuxDeMots ou PtiClic, proposent, valident et enrichissent des relations entre termes. Cette méthode permet d'obtenir des données à la fois massives, variées et validées par le consensus, ce qui garantit leur qualité. D'autres initiatives existent, telles que Verbosity ou Phrase Detectives, mais JeuxDeMots se démarque par la longévité du projet, la taille de sa base et la diversité des relations collectées.

3. L'inférence dans les graphes de connaissances : modèles et algorithmes

L'un des enjeux majeurs des graphes de connaissances est la capacité à inférer de nouvelles relations à partir de celles déjà existantes. Plusieurs approches ont été développées :

- **Modèles vectoriels** : des méthodes comme TransE ou DistMult représentent les entités et les relations sous forme de vecteurs dans un espace latent, permettant de prédire de nouveaux liens par des opérations algébriques.
- **Algorithmes de parcours de graphe** : des techniques comme le Path Ranking Algorithm explorent les chemins entre nœuds pour identifier des relations implicites ou manquantes.
- **Méthodes d'analogie ou de raffinement** : elles exploitent des motifs récurrents ou des schémas relationnels pour compléter ou corriger le graphe.

Dans le contexte de JeuxDeMots, des travaux antérieurs ont exploré l'inférence par analogie, la détection de motifs, ou encore l'utilisation de la validation croisée entre joueurs pour renforcer la fiabilité des relations. L'inférence sémantique permet ainsi d'enrichir le réseau, de détecter des incohérences ou de proposer des explications pour des relations complexes.

4. La détection de triangles dans les graphes : un outil d'analyse et d'inférence

La détection de triangles est un problème classique en théorie des graphes, avec des applications variées en analyse de réseaux sociaux, en biologie, ou en modélisation des connaissances. Dans un graphe sémantique, les triangles – c'est-à-dire des cycles de trois nœuds reliés entre eux – sont particulièrement intéressants car ils révèlent des motifs de redondance, de cohérence ou de complémentarité entre concepts. Des algorithmes efficaces, comme ceux basés sur les jointures multiples ou l'énumération de motifs, permettent de détecter et de compter les triangles même dans des graphes massifs. Dans le domaine des graphes de connaissances, ces structures ont été utilisées pour :

- Compléter des graphes incomplets en suggérant des relations manquantes
- Détecter des incohérences ou des biais dans la structuration des données
- Générer des explications pour des relations complexes, en s'appuyant sur des cycles interprétables

Dans JeuxDeMots, l'étude des triangles sémantiques permet non seulement de valider la cohérence du réseau, mais aussi d'identifier des zones denses en relations, propices à l'inférence et à l'explication.

5. L'intelligence artificielle explicable (XAI) : vers des systèmes transparents et interprétables

L'essor de l'intelligence artificielle explicable (XAI, « eXplainable Artificial Intelligence ») répond à un besoin croissant de transparence et de confiance dans les systèmes automatisés. Dans de nombreux domaines sensibles (santé, droit, éducation), il est crucial de pouvoir expliquer les décisions prises par une IA.

Les triangles sémantiques, en tant que structures simples et interprétables, constituent un support idéal pour générer des explications sur la façon dont une IA établit des liens entre concepts ou prend des décisions. Par exemple, si un système recommande un mot ou une relation, il peut justifier son choix en s'appuyant sur un triangle sémantique reliant les concepts concernés. Cette capacité d'explication renforce la confiance des utilisateurs et facilite l'adoption des technologies d'IA dans des contextes variés.

En résumé, l'étude des réseaux lexico-sémantiques, et plus particulièrement de Jeux De Mots, s'appuie sur des avancées majeures en linguistique computationnelle, en théorie des graphes et en intelligence collective.

La construction collaborative de ressources linguistiques via des jeux avec un but a permis d'obtenir une base de données d'une richesse et d'une qualité exceptionnelles pour le français. L'analyse des triangles sémantiques, quant à elle, offre de nouveaux leviers pour l'inférence, la validation et l'explication des relations au sein du réseau. L'intégration de ces approches dans le champ de l'IA explicable ouvre des perspectives prometteuses, tant pour la recherche fondamentale que pour les applications concrètes : amélioration des chatbots, enrichissement automatique des bases de connaissances, ou encore développement de systèmes d'aide à la décision transparents et interprétables. Ainsi, l'étude des triangles sémantiques dans JeuxDeMots s'inscrit pleinement dans une dynamique d'innovation au service de la compréhension du langage et de la démocratisation de l'intelligence artificielle.

PARTIE 2 : METHODOLOGIE

1. Organisation du projet

Dès le lancement du projet, nous avons accordé une importance particulière à la mise en place d'une organisation rigoureuse pour en assurer le bon déroulement. La question du choix des outils collaboratifs s'est rapidement imposée. Bien que GitHub soit la solution classique pour ce type de projet, nous avons initialement écarté cette option en raison de notre manque de familiarité avec cette plateforme. Nous avons finalement appris à l'utiliser dans le cadre d'autres cours, mais pour ce projet spécifique, nous avons estimé que d'autres outils seraient plus adaptés à nos besoins immédiats.

Nous avons opté pour deux solutions alternatives :

- **Google Drive** Fourni avec google, facile d'utilisation et nous permet de partager nos travaux et dossiers assez facilement.
- **WhatsApp** a quant à lui servi de canal de communication principal. Sa rapidité et son accessibilité en ont fait un outil indispensable pour les échanges quotidiens et la coordination des tâches en temps réel.

La diversité des profils au sein de l'équipe a constitué à la fois un défi et une richesse. Si l'environnement informatique était nouveau pour certains, l'entraide et la complémentarité des compétences ont permis à chacun de progresser. Khaled, plus expérimenté en programmation, a joué un rôle moteur dans la structuration du projet et l'initiation aux outils techniques. L'intelligence artificielle, que nous avons mobilisée pour la génération de code, la documentation et la résolution de problèmes, a également accéléré notre montée en compétence collective.

La répartition des rôles s'est faite naturellement : Khaled et Hajar se sont concentrés sur le développement et l'implémentation des algorithmes, tandis que Lynna et Sofia ont pris en charge la rédaction du rapport et la synthèse des résultats. Toutefois, cette organisation est restée flexible, chaque membre étant encouragé à s'impliquer dans toutes les dimensions du projet afin de garantir une compréhension globale et partagée des enjeux et des solutions mises en œuvre.

2. Présentation de l'API JeuxDeMots

Pour ce projet, nous avons eu la chance de disposer d'une API dédiée permettant d'interroger l'intégralité des données contenues dans le graphe sémantique de JeuxDeMots. Cette API

constitue le socle technique de notre travail, offrant un accès direct et structuré aux concepts (nœuds) ainsi qu'aux relations (arcs) qui les lient.

L'API expose un total de huit endpoints distincts, chacun correspondant à une fonctionnalité précise. Parmi ces huit points d'accès, seuls trois endpoints sont essentiels à notre étude :

L'endpoint de récupération des relations sortantes :

to/{node2_name}

Cet endpoint permet, à partir d'un mot donné, d'obtenir l'ensemble des relations sortantes (outgoing relations), c'est-à-dire les liens qui partent d'un nœud vers d'autres nœuds dans le graphe.

L'endpoint de récupération des relations entrantes :

from/{node1_name}

À l'inverse du précédent, cet endpoint permet de récupérer toutes les relations entrantes (incoming relations) vers un nœud spécifique.

Ces liens représentent les concepts qui pointent vers le nœud cible dans le graphe.

L'endpoint de récupération des relations entre deux mots :

from/{node1_name}/to/{node2_name}

Enfin, cet endpoint permet de trouver s'il existe un lien sémantique direct entre deux nœuds spécifiés.

Cela permet d'explorer les connexions précises qui existent, par exemple, entre feu et fumée, ou entre eau et pluie.

Si une relation existe, l'API retourne sa nature (causalité, association, synonymie, etc.).

Ces trois points d'accès sont cruciaux dans notre méthodologie car ils nous permettent de naviguer efficacement dans le graphe, de détecter les schémas d'inférence, et d'identifier les triangles recherchés.

3. Scripts python et algorithmes utilisés

3.1. Obstacles rencontrés et solutions

Pour ce projet, nous avons fait le choix stratégique d'utiliser Python comme langage de programmation. Bien que d'autres options techniques aient été envisageables (comme PHP ou JavaScript), notre préférence s'est naturellement portée sur Python pour plusieurs raisons : sa lisibilité, sa richesse en bibliothèques adaptées au traitement de données, et surtout notre maîtrise par rapport aux autres langages.

Un script simpliste pour explorer les triangles lexicaux

Notre premier script Python avait un objectif simple : identifier tous les triangles lexicaux possibles à partir d'un mot donné. Concrètement, il s'agissait de trouver des triplets de mots interconnectés selon des relations sémantiques prédéfinies.

Néanmoins, cette simplicité de façade cachait des défis inattendus. Nos premières tentatives se sont soldées par des échecs, principalement dus à une méconnaissance initiale de certaines subtilités algorithmiques. Après plusieurs itérations et ajustements, nous avons finalement obtenu un script fonctionnel, capable de générer ces triangles de manière cohérente.

Problèmes rencontrés et optimisations nécessaires

Une fois le script opérationnel, deux problèmes majeurs sont rapidement apparus :

1. Temps d'exécution excessif

Pour y remédier, nous avons implémenté une limite arbitraire sur le nombre de triangles retournés et nous avons introduit une gestion de cache permettant d'optimiser les requêtes envoyées. Bien que basique, cette solution a permis de garder des temps de réponse acceptables lors des tests.

2. Pertinence des triangles générés

Certaines relations entre mots avaient un type de relation incorrecte ou négligeable, nous avons donc identifié les types de relation à ignorer avec M. Lafourcade, ces types indiquant une connexion peu pertinente ou contre-intuitive. Nous avons donc introduit un filtre basé sur le type des relations, éliminant automatiquement les triangles dont au moins une relation présentait faisant partie de la liste des types de relation à ignorer.

En conséquence, notre script Python a connu plusieurs améliorations majeures. La première concerne sa flexibilité accrue : il permet désormais de filtrer les résultats en fonction du poids et des types des relations et de limiter le nombre de triangles générés. Cette fonctionnalité s'est révélée indispensable pour optimiser les temps de calcul et affiner la pertinence des résultats. L'autre avancée notable réside dans l'automatisation du processus. Notre script puise désormais directement dans la base de données de l'API, pour les nœuds B et C, ce qui élimine tout biais de sélection et enrichit considérablement la diversité des résultats. Cette approche systématique nous a permis d'obtenir des données plus représentatives et reproductibles.

Nous avons accordé une attention particulière à la visualisation des résultats. L'intégration de bibliothèques comme nous permet désormais de générer des graphes interactifs et des histogrammes qui mettent en lumière la distribution des poids des relations. Ces représentations graphiques se sont avérées précieuses pour interpréter les données et valider nos hypothèses de manière intuitive.

3.2. Stratégie de stockage des données

L'aboutissement de nos différents scripts a soulevé une question récurrente : quel système de stockage adopter pour pérenniser et exploiter nos résultats ? Deux options s'offraient à nous : une solution légère avec un simple fichier CSV, ou une approche plus structurée via une base de données SQLite. Ce choix stratégique a donné lieu à une réflexion approfondie sur nos besoins immédiats et nos perspectives d'évolution.

Phase exploratoire : la simplicité du CSV

Dans les premières phases du projet, le format CSV s'est imposé comme une solution idéale. Ses avantages étaient multiples : mise en œuvre immédiate, facilité de lecture et de manipulation manuelle, et compatibilité avec l'ensemble de nos outils d'analyse. Ce format minimaliste répondait parfaitement à nos besoins initiaux de prototypage rapide.

Passage à une architecture plus robuste avec SQLite

Une fois le script stabilisé et les algorithmes validés, nous avons opéré une migration vers une base de données SQLite tout en gardant l'infrastructure csv fonctionnelle pour les analyses statistiques. Cette évolution répondait à plusieurs objectifs : centralisation des données et préparation pour des traitements futurs plus complexes. La base sert actuellement de référentiel pour l'ensemble des triangles détectés et des relations extraites, offrant une structure claire et normalisée.

Cependant, notre utilisation actuelle de la base reste volontairement limitée. Toutes nos analyses statistiques (calculs de fréquences, moyennes, distributions) continuent de s'exécuter en mémoire vive, sans recourir aux capacités de requêtage de SQLite. Ce choix se justifie par deux facteurs :

1. Les volumes de données traités restent modestes, permettant des calculs en temps réel sans impact perceptible sur les performances.
2. Notre workflow actuel privilégie la rapidité d'itération sur l'optimisation des ressources.

Perspectives d'exploitation future

L'implémentation de la base de données, bien qu'encore sous-utilisée, ouvre la voie à de nombreuses possibilités d'enrichissement :

- **Requêtes avancées** pour isoler des motifs spécifiques de triangles ou des relations particulières
- **Croisement de données** avec d'autres sources lexicales grâce aux jointures
- **Développement d'interfaces** analytiques plus sophistiquées (tableaux de bord, visualisations interactives)
- **Alimentation de modèles** de graphes pour des analyses sémantiques approfondies
- Manipulation pour analyse de nombre énorme de triangles qui seront stockés au fur et à mesure dans la BD.

Bilan et feuille de route

Notre approche progressive - du CSV expérimental à la base de données structurée - s'est révélée payante. Il a joué son rôle de solution transitoire idéale pendant la phase de recherche et développement. À mesure que le projet gagnera en maturité et que les volumes de données augmenteront, nous prévoyons :

1. D'exploiter pleinement le potentiel de requêtage de SQLite
2. D'implémenter des indexations stratégiques pour accélérer les recherches
3. De développer des procédures stockées pour automatiser les analyses récurrentes

Cette évolution reflète parfaitement notre philosophie de développement : commencer simple pour valider les concepts, puis renforcer progressivement l'infrastructure en fonction des besoins réels. La base de données, aujourd'hui en sommeil, constitue ainsi une réserve de potentiel prête à être activée lorsque le projet entrera dans sa phase de production à grande échelle.

3.3. Structure et fonctionnement du script

Structure et Architecture du Projet

Le projet est conçu comme un système modulaire sophistiqué, où chaque composant joue un rôle crucial dans l'analyse des triangles de relations sémantiques. L'architecture repose sur cinq modules principaux qui forment un pipeline de traitement complet : `main.py` pour l'orchestration, `api_client.py` pour l'interaction avec l'API JDM, `triangle_finder.py` pour l'algorithme de recherche, `triangle_db.py` pour la persistance des données, et `visualizer.py` pour l'analyse visuelle.

La Chasse aux Triangles : Un Algorithme Intelligent

Le cœur du système réside dans sa capacité à identifier des triangles de relations dans le graphe sémantique. Le processus commence par l'initialisation du système, où le programme crée l'infrastructure nécessaire et configure les paramètres de recherche. Ces paramètres incluent le poids minimal des relations acceptables et le nombre maximum de triangles par nœud, permettant ainsi une recherche ciblée et efficace. La recherche des triangles suit une approche systématique et optimisée. Pour chaque nœud initial, le système explore les relations en profondeur, identifiant les triangles valides selon des critères prédéfinis. Cette exploration est guidée par un algorithme qui vérifie la présence de trois types de relations (R1, R2, R3) formant un triangle, tout en respectant les contraintes de poids et de nombre.

Système de Visualisation et Analyse

Un aspect particulièrement innovant du projet est son système de visualisation avancé. Le module `visualizer.py` transforme les données brutes en représentations visuelles riches et

informatives. Ces visualisations permettent d'analyser les patterns de relations sous différents angles :

- La structure du réseau de triangles révèle les connexions complexes entre les nœuds
- Les distributions de types de relations mettent en évidence les patterns dominants
- L'analyse des poids des relations permet d'identifier les relations les plus significatives
- L'étude des paires de relations R2-R3 (qui forme le chapeau du triangle) avec les relation R1 (qui forme la base du triangle) offre une vue détaillée des combinaisons de relations

Optimisation et Performance

Le système intègre plusieurs mécanismes d'optimisation pour garantir des performances optimales. Un système de cache sophistiqué réduit les appels redondants à l'API, tandis que la base de données SQLite assure une persistance efficace des données. La classe TriangleDatabase gère de manière centralisée toutes les opérations liées aux données. Elle s'appuie sur une base de données SQLite structurée autour de trois tables essentielles :

- triangles : Enregistre chaque triangle détecté avec ses nœuds, types de relations, poids et date de création
- processed_nodes : Suit l'état de traitement des nœuds, permettant d'éviter les doublons et de reprendre le traitement après une interruption
- statistics : Stocke les métriques de chaque session (temps d'exécution, nombre de triangles, seuils, etc.)

Cette architecture permet une gestion efficace des données tout en offrant la possibilité de reprendre le traitement à tout moment et d'analyser les résultats de manière approfondie.

Interface et Expérience Utilisateur

L'interface utilisateur, bien que basée sur la ligne de commande, offre une expérience intuitive et riche en fonctionnalités. Les utilisateurs peuvent facilement configurer les paramètres de recherche, suivre la progression en temps réel, et accéder à des rapports détaillés sur les résultats. Cette interface est conçue pour être à la fois accessible aux débutants et suffisamment puissante pour les utilisateurs avancés.

Technologies et Bibliothèques

Le projet s'appuie sur un écosystème robuste de bibliothèques Python, chacune apportant des fonctionnalités essentielles. networkx gère la manipulation des graphes, matplotlib et seaborn créent des visualisations sophistiquées, pandas traite efficacement les données, et requests assure une communication fiable avec l'API. Cette combinaison de technologies permet de créer un système puissant et flexible pour l'analyse des relations sémantiques. Cette implémentation représente une approche efficace pour l'analyse des relations sémantiques, combinant des algorithmes sophistiqués avec des visualisations intuitives pour offrir une compréhension approfondie des patterns de relations dans le graphe.

Passons maintenant à l'explication de l'intégration de la base de données :

Le script **triangle_db.py** a été conçu pour assurer la gestion structurée et centralisée des données du projet. Il fournit une interface dédiée pour l'enregistrement, le suivi et l'organisation des triangles détectés ainsi que des relations extraites, en s'appuyant sur la base de données SQLite mise en place.

La classe principale, **TriangleDatabase**, regroupe l'ensemble des opérations nécessaires à la manipulation de ces données : création des tables, insertion des résultats, gestion de la progression et sauvegarde des informations utiles pour d'éventuelles analyses futures.

La base de données SQLite « triangles.db » est structurée autour de trois tables principales :

La table triangles :

Elle enregistre chaque triangle détecté, avec :

- Les trois nœuds impliqués (A, B, C)
- Les types de relations entre ces nœuds
- Les poids associés à chaque relation
- La date de création du triangle

Cela permet de conserver un historique complet et d'éviter les doublons.

La table processed_nodes :

Elle garde la trace des nœuds déjà traités, avec leur statut (en cours ou terminé) et la date de traitement.

Elle permet :

- D'éviter de retraiter les mêmes nœuds
- De reprendre le traitement automatiquement en cas d'interruption du programme

Et enfin la table statistics :

Cette table stocke des informations globales sur chaque session d'exécution :

- Temps total d'exécution
- Nombre de triangles trouvés
- Nombre de relations rejetées
- Seuils utilisés pour les poids
- Statistiques sur les types de relations et les poids moyens

Ces données sont essentielles pour l'analyse et l'optimisation du processus.

Concernant l'initialisation et la création des tables. A l'instanciation la classe crée automatiquement les tables nécessaires si elles n'existent pas.

4. Utilisation de l'IA dans le projet

4.1. Pourquoi utiliser l'IA ?

Adoption des IA génératives : une nécessité stratégique

Initialement, notre approche excluait l'utilisation d'IA génératives, par souci d'autonomie et de maîtrise totale du code. Cependant, face à la complexité croissante du projet, nous avons réalisé que ces outils pouvaient nous faire gagner un temps précieux tout en nous permettant de nous concentrer sur des aspects critiques.

Choix des outils et méthodologie

Pour intégrer l'IA de manière efficace, nous avons sélectionné trois plateformes :

- ChatGPT
- DeepSeek
- Cursor

Notre méthode :

D'abord, nous partons toujours d'une base de code que nous avons nous-mêmes développée, afin de maintenir une cohérence initiale. Ensuite, plutôt que de demander à l'IA de tout reconstruire, nous lui soumettons des demandes d'ajustements progressifs, comme l'optimisation d'une fonction ou la clarification d'un algorithme. Enfin, chaque modification était comparée entre les différentes IA pour identifier la solution la plus pertinente. Nous interrogeons également les modèles sur leur propre raisonnement afin de détecter d'éventuelles incohérences.

Parmi les trois solutions testées, Cursor s'est révélé le plus performant. Son intégration fluide avec les environnements de développement, sa compréhension avancée du contexte du projet et ses suggestions précises en ont fait notre outil privilégié.

Si l'IA générative a été un accélérateur, son succès repose sur une utilisation raisonnée et supervisée, où notre expertise a guidé et validé chaque étape du processus.

4.2. Les prompts

Au cours de cette année, nous avons vu lors de différents enseignements plusieurs façon d'effectuer un bon prompt. La méthode que nous avons le plus utilisé est la méthode "**MOTTIF-TV**". Cette méthode consiste en huit éléments clés : MOI, OBJECTIF, TOI, TÂCHE,

INSTRUCTIONS, FORMAT, TON, VALIDATION. Voici des extraits de notre cours sur cette méthode qui vous permettront de mieux la comprendre.

LE MODELE MOTTIF-TV

Le modèle MOTTIF-TV consiste à structurer un prompt en renseignant les éléments clés suivants :

M OI	→	Identité de la personne qui formule la demande et contexte de la demande. Permet au modèle d'adapter sa réponse en fonction des connaissances, du niveau d'expertise ou des besoins particuliers de l'utilisateur.	"Je suis étudiant en management"
O BJECTIF	→	But ou résultat souhaité. Cela aide à orienter le modèle vers ce que l'on espère obtenir comme réponse.	"Je veux comprendre comment mieux prompter"
T OI	→	Le "Toi" spécifie la fonction, le rôle que le modèle doit remplir lorsqu'il répond. Cela aide à guider la façon dont le modèle aborde le sujet.	"Tu es expert en conception de prompt"
T ÂCHE	→	La tâche représente ce que le modèle doit faire en réponse au prompt. Cela peut inclure des actions comme expliquer, comparer, recommander...	"Fais une synthèse des éléments qu'un bon prompt doit contenir"
I NSTRUCTIONS	→	Consignes spécifiques que l'on va donner au modèle. On pourra indiquer des étapes avec des stops si la tâche est complexe. Les instructions peuvent indiquer les priorités, pour que la réponse se concentre sur l'essentiel (Exemple : privilégier les recommandations pratiques plutôt que théoriques).	"Tu vas d'abord lister les éléments clés d'un prompt. Stop et valide."
F ORMAT	→	Manière dont la réponse doit être structurée ou présentée. Cela peut inclure des aspects comme le type de réponse (liste, tableau, paragraphe, etc), la longueur ou la structure du texte. Le format peut préciser des contraintes (Exemple : Répondre en moins de 150 mots, éviter les termes techniques...)	"Réponds sous forme de liste à puces"
T ON	→	Attitude ou style de communication que le modèle doit adopter dans sa réponse, comme formel, informel, enthousiaste, neutre, etc.	"Sois clair et précis"
V ALIDATION	→	Consiste à demander au modèle de reformuler ou de confirmer ce qu'il a compris. Cela permet de vérifier que le modèle a bien saisi nos attentes et qu'il posera des questions de clarification si nécessaire.	"Est-ce que tu as bien compris ma demande ? Pose moi des questions si nécessaire"

+ REMARQUE



Dans certains cas, les contenus générés par le modèle ne nous sont pas destinés. Il est donc important d'indiquer à qui ou à quoi s'adresse la réponse. Exemple : Un message destiné à un recruteur.



Exemple de prompt :

Trouver pour chaque chapeau le nombre de base négatives :

M – Moi

Je suis étudiant en Master 1 Informatique. J'apprends à utiliser les API REST et à faire de l'analyse de graphes en Python.

O – Objet

Je veux développer un script Python capable de récupérer des relations entre entités via une API HTTP (méthode GET) et de détecter des relations triangulaires dans ces données (des triplets d'entités toutes reliées entre elles).

T – Toi

Tu es une IA spécialisée en programmation Python, apte à produire du code fonctionnel, bien structuré, commenté, et conforme aux bonnes pratiques.

I – Instruction

Créer un script Python qui :

- Effectue une requête GET sur une API (tu peux simuler les données si besoin),
- Représente les relations sous forme de graphe (avec networkx),
- Détecte toutes les relations triangulaires (cliques de taille 3),
- Affiche ou retourne la liste de ces triangles.

F – Forme

Le code doit être clair, bien commenté, utilisable dans un contexte académique. Utilise les bibliothèques standard comme requêtes pour l'appel API et networkx pour l'analyse des graphes. Si tu simules les données, fais-le avec un exemple JSON bien structuré.

5. Validation et test

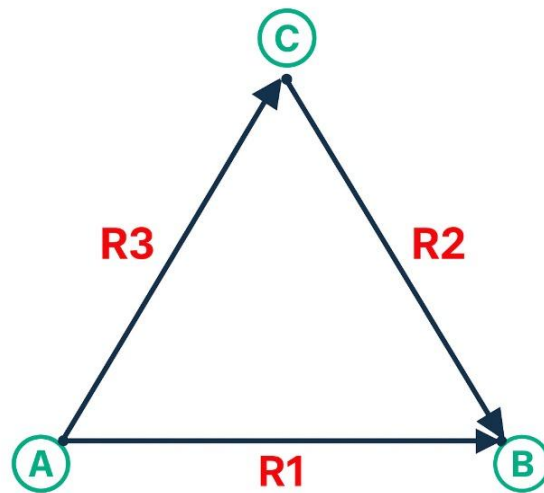
5.1. Les triangles

Définition et enjeux des triangles sémantiques

Un triangle sémantique est une structure composée de trois nœuds (termes) interconnectés par des relations directes ou indirectes, formant ainsi un cycle logique. Par exemple :

- Le terme A est lié au terme B
- Le terme C est lié à B
- Le terme A est également lié à C

Ces trois relations forment alors un triangle cohérent, à condition que les liens entre les termes aient un sens sémantique valide comme dans la figure ci-dessous :



Première phase : collecte et validation des triangles

L'objectif initial de cette recherche était d'identifier un maximum de triangles sémantiques afin de constituer un échantillon statistiquement significatif. Pour cela, il était essentiel de :

1. **Garantir la cohérence des triangles** : chaque triplet devait être composé de trois nœuds distincts reliés de manière logique.
2. **Éliminer les relations non pertinentes** : certains triangles présentaient des incohérences sémantiques ou incluaient des relations avec un poids négatif ($w < 0$), indiquant une connexion faible ou erronée.

Optimisations et corrections apportées

Pour résoudre ces problèmes, plusieurs améliorations ont été implémentées :

1. Filtrage automatique des relations

Une fonction (`filter_relations()` dans `api_client.py`) a été ajoutée pour ne conserver que les relations dont le poids w est strictement positif ($w \geq 0$). Ce filtrage a permis d'éliminer les liaisons peu fiables et de réduire le bruit dans les résultats.

2. Tests progressifs sur un sous-ensemble de nœuds

Le script a d'abord été exécuté sur un nombre limité de termes pour faciliter la vérification manuelle. Cette approche a permis de détecter rapidement des incohérences dans la logique de parcours et d'ajuster l'algorithme avant de passer à une analyse à grande échelle.

3. Gestion des doublons dans la base de données

Même si la logique de sélection des relations dans `triangle_finder.py` minimise naturellement le risque de triangle, nous avons décidé pour la version avec la base de données d'ajouter une contrainte d'unicité sur les triangles. Cette gestion des doublons est assurée dans le fichier `triangle_db.py`, où la table des triangles est créée avec la commande SQL `UNIQUE(node_a, node_b, node_c, relation_a_to_b, relation_c_to_b, relation_a_to_c)`. De plus, l'insertion des triangles se fait avec la commande `INSERT OR IGNORE`, ce qui permet de ne pas enregistrer un triangle déjà existant dans la base. Ainsi, chaque triangle n'est stocké qu'une seule fois, même s'il est détecté plusieurs fois au cours de l'exécution, ce qui garantit l'absence de doublons exacts dans les résultats finaux.

Ces optimisations ont permis d'obtenir un jeu de données fiable et exploitable pour des analyses linguistiques ultérieures. La combinaison d'un filtrage rigoureux et d'une gestion efficace des doublons a considérablement amélioré la qualité des résultats.

PARTIE 3 : RESULTATS ET EVALUATION

1. Statistiques globales

1.1. Explications

À la fin de l'exécution, le script affiche une synthèse chiffrée qui permet de dresser un **bilan global des résultats obtenus** pour une configuration donnée (seuil de poids, nombre de triangles par nœud, etc.).

Les volumes de triangles détectés varient selon les paramètres choisis et les mots explorés, mais on observe généralement que :

- la majorité des triangles détectés sont composés de relations **fortement pondérées** (souvent supérieures à 10), ce qui confirme l'utilité du filtrage par poids.
- certains types de relations sont **largement sur-représentés**, notamment ceux exprimant l'appartenance ou la similarité.

Les statistiques générées permettent ainsi de **valider les grandes tendances** du graphe sémantique, d'**identifier les types de relations dominants**, et de **repérer les éventuelles anomalies**, comme des distributions de poids trop dispersées ou des déséquilibres relationnels.

Cette vision macro des résultats complète les observations plus fines réalisées dans les tests itératifs (voir partie 5.2), et confirme que la stratégie choisie pour filtrer, extraire et structurer les données donne des résultats globalement cohérents et exploitables.

1.2. Test effectués

Afin d'obtenir les statistiques présentées plus loin, nous avons procédé à 3 exécutions principales du script, chacune portant respectivement sur **10 mots, 11 mots et 14 mots de la node A**, avec une **limite fixée à 10 000 triangles par mot**. Le dernier test que nous avons fait est un peu particulier, nous avons fourni une liste de 13 mots, mais cette fois nous n'avons pas mis de limite de triangles par mot. Ces données ont été fusionnées avec tous les tests faits précédemment afin de nous permettre d'avoir une grosse base de données qui nous a permis d'avoir des statistiques et graphiques beaucoup plus précis.

Pour chaque test nous avons mis le poids minimum acceptable à -999 afin d'avoir également les relations négatives.

Le développement du script complet ayant pris du temps, nous avons dans un premier temps limité nos tests à **100 triangles par mot**, afin de nous assurer que la logique de détection

fonctionnait bien et d’avoir une vue rapide sur les structures générées. Cependant, nous avons rapidement constaté que ces résultats étaient **trop restreints pour permettre une analyse fiable ou généralisable**.

Une fois le script finalisé, nous avons pu lancer des exécutions plus ambitieuses :

- **Premier test** sur les mots en node A : voiture, fauteuil, cercueil, panda, internet, cahier, lampe, étoile, méduse, et vampire
- **Deuxième test** sur les mots en node A : table, master, projet, informatique, examen, télévision, faire, portable, porte
- **Troisième test** sur les mots en node A : arbre, chaine, chaise, feuille, fleur, fromage, fruit, lait, mot, pain, poisson, sac, viande, et ville
- **Quatrième test** sur les mots en node A : table, ciel, bleu, sourcil, chemise, avion, chat, informatique, vie, eau, maison, voiture, et crème

Ces groupes de mots ont été choisis pour leur diversité lexicale et thématique, afin d’évaluer le **comportement du graphe dans différents champs sémantiques**. Grâce à ces tests, nous avons pu générer des volumes de triangles suffisants pour extraire des statistiques significatives et des motifs relationnels récurrents.

1.3. Résultats

À l’issue de l’exécution complète du premier script, un total de **100 006 triangles** a été détecté, sans qu’aucune relation ne soit rejetée (seuil de poids minimal fixé à -999). Avec une limite définie à **10 000 triangles maximum par nœud**, permettant de cadrer l’extraction dans des conditions maîtrisées.

Les types de relations identifiés dans les triangles sont très diversifiés. Certains types sont très fréquents : **r_hypo (type 8), r_isa (type 6), r_syn (type 5), r_has_part (type 9), r_lieu_1 (type 28), r_lieu_action (type 30), r_domain (type 3)**... pour n’en citer que quelques-uns.

D’autres types apparaissent de manière plus marginale (ex. : **type 139 avec 4 triangles, type 84 avec seulement 1 occurrence**), ce qui montre à la fois la **richesse** et l’hétérogénéité des relations présentes dans la base JeuxDeMots.

Cette répartition confirme l’intuition que certains types de relations — probablement plus **structurantes** (appartenance, caractérisation, synonymie, etc.) — sont à l’origine d’un **grand nombre de motifs triangulaires**, tandis que d’autres restent plus **anecdotiques**.

Ces statistiques globales, combinées aux analyses structurelles menées en amont, permettent de conclure que la stratégie d’extraction adoptée est efficace pour identifier des motifs significatifs, tout en restant ouverte à une grande diversité de relations.

Une deuxième exécution du script a été menée sur un ensemble de mots différents, avec les mêmes paramètres de configuration (seuil de poids minimal à -999 et limite de 5 000 triangles par nœud). Pour un total de **45 007 triangles détectés**.

La troisième, elle, a été menée sur un autre ensemble de mot encore. Avec toujours les mêmes paramètres de configuration. Au final, un total de **131 593 triangles détectés**.

Répartition des types de relations

Les résultats confirment certaines tendances observées lors du premier test, tout en révélant de nouvelles relations dominantes. On note notamment :

- **r_hypo (type 8)** avec **38 272 triangles**, de loin le plus fréquent dans cette série
- Suivi par **r_lieu_1 (type 28)** avec **10 149 triangles**, **r_carac (type 17)** avec **9 418 triangles**, **r_isa (type 6)** avec **3 872 triangles**, **r_syn (type 5)** avec **2 137 triangles**, et **r_holo (type 10)** avec **12 763 triangles**.

Cette répartition renforce l'idée que certains types de relations s'imposent comme **structurants dans le graphe lexical**, formant les **fondations des motifs triangulaires les plus fréquents**.

À l'inverse, plusieurs types apparaissent de manière très marginale, comme **le type 54 (1 triangle)** ou **le type 139 (6 triangles)**.

Cela illustre encore une fois **l'hétérogénéité sémantique de la base JeuxDeMots**, où des relations très courantes coexistent avec des **liens rares ou spécialisés**, souvent trop peu fréquents pour jouer un rôle significatif dans l'organisation du graphe.

Et enfin, le dernier test a toujours à peu près les mêmes configurations, la seule différence réside dans le fait qu'il n'y a pas de limite pour chaque triangle. Nous avons fusionné tous nos résultats. C'est avec cette base là que nous allons vraiment analyser les résultats.

2. Visualisation des triangles détectés

2.1. Explication générale

Pour faciliter l'analyse, des représentations graphiques sont automatiquement générées à la fin de l'exécution du script, à l'aide du module Visualizer.

Plusieurs visualisations sont produites :

- Une représentation du graphe avec les relations entre concepts (network.png)

- Une distribution des types de relations présentes dans les triangles (relation_stats.png)
- Un histogramme des poids des relations (weight_dist.png)
- Une répartition des paires de types de relations (R2-R3) selon chaque type R1 (relation_type_pairs.png).
- Le nombre de paire R2-R3 différentes par type R1 (hats_per_r1.png)
- Le nombre de paires avec des poids négatifs par type R1 (negative_hats_per_r1.png)
- Le nombre de relations R1 négatives par paire R2-R3 (negative_r1_distribution.png)

Ces graphiques sont enregistrés automatiquement dans un dossier nommé “Data_graph” situé dans le répertoire de travail. Le script propose également à l'utilisateur d'explorer dynamiquement des sous-ensembles du graphes, soit par type de relation R1, soit par couple de nœuds A-B spécifiques, afin de mieux comprendre les structures locales.

2.2. Explications des différents graphiques

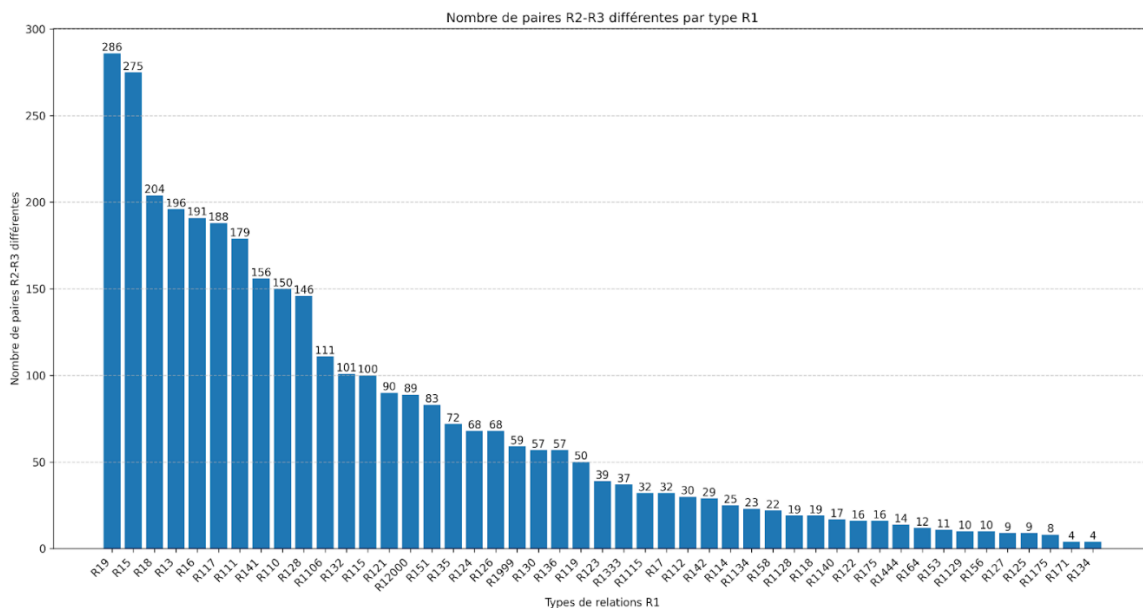
Afin de vous faciliter la tâche, nous allons maintenant vous expliquer les graphiques qui nous serviront le plus lors de nos recherches, ils nous sont fournis par notre script python.

Notre script fourni plusieurs diagrammes dont 4 sont les plus importants :

- Nombre de chapeaux par base
- Nombre de bases par chapeau
- Nombre de chapeaux à poids négatif par base
- Nombre de base à poids négatif par chapeau

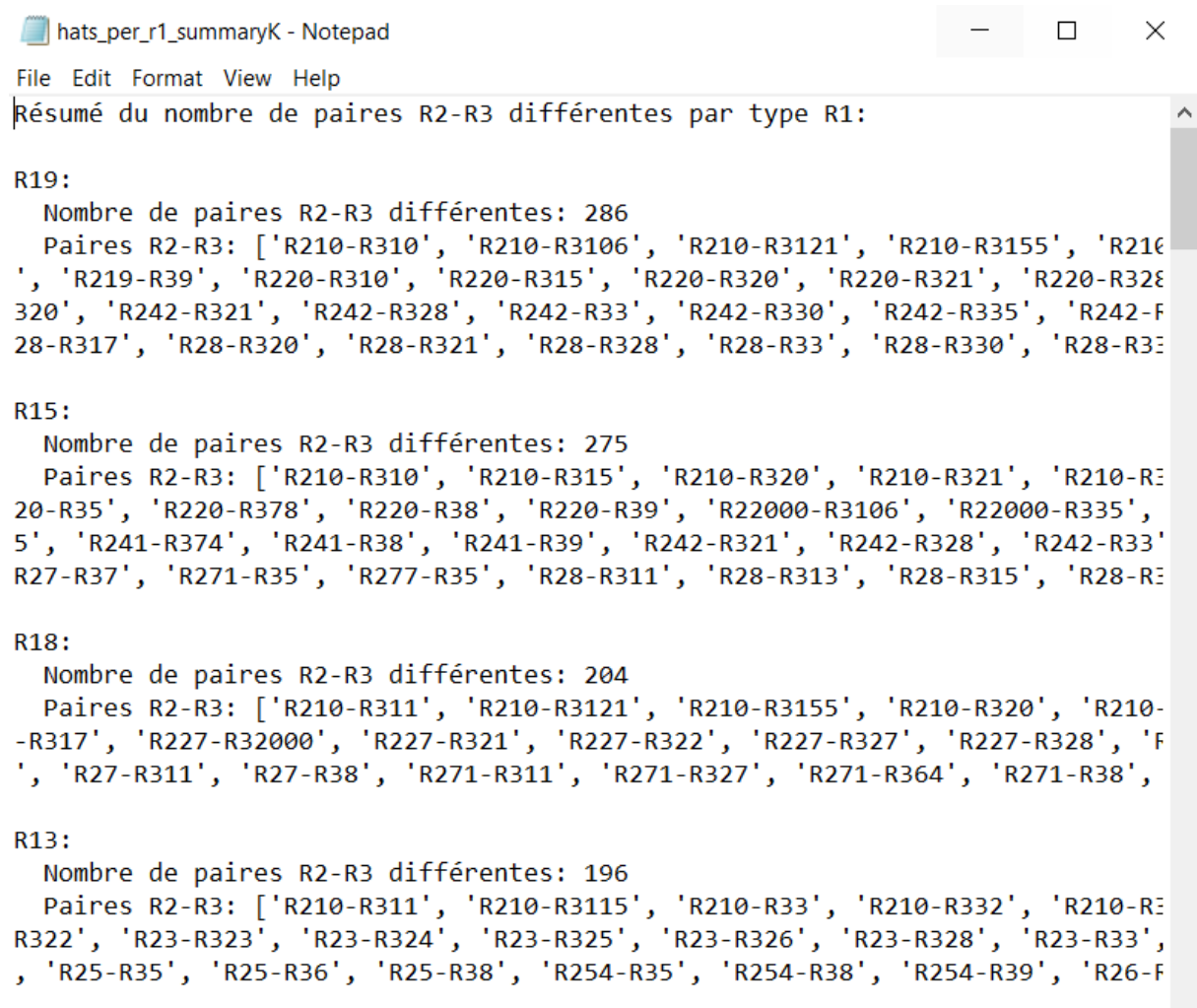
Voici des exemple et explication sur chaque graphiques :

Nombre de chapeau par base :



Ce graphique correspond au nombre de chapeaux (R2-R3) trouver par type de base (R1).
Il nous permettra d'identifier les relations de la base du triangle (R1) qui sont les plus propice à avoir plus de chapeau du triangle (R2-R3) et donc on identifie les bases les plus pertinentes.

Détails des valeurs :



```

hats_per_r1_summaryK - Notepad
File Edit Format View Help
Résumé du nombre de paires R2-R3 différentes par type R1:

R19:
  Nombre de paires R2-R3 différentes: 286
  Paires R2-R3: ['R210-R310', 'R210-R3106', 'R210-R3121', 'R210-R3155', 'R210-R317', 'R219-R39', 'R220-R310', 'R220-R315', 'R220-R320', 'R220-R321', 'R220-R328', 'R220-R328', 'R242-R321', 'R242-R328', 'R242-R33', 'R242-R330', 'R242-R335', 'R242-R337', 'R28-R320', 'R28-R321', 'R28-R328', 'R28-R33', 'R28-R330', 'R28-R337']

R15:
  Nombre de paires R2-R3 différentes: 275
  Paires R2-R3: ['R210-R310', 'R210-R315', 'R210-R320', 'R210-R321', 'R210-R325', 'R220-R378', 'R220-R38', 'R220-R39', 'R22000-R3106', 'R22000-R335', 'R22000-R335', 'R241-R374', 'R241-R38', 'R241-R39', 'R242-R321', 'R242-R328', 'R242-R33', 'R27-R37', 'R271-R35', 'R277-R35', 'R28-R311', 'R28-R313', 'R28-R315', 'R28-R317']

R18:
  Nombre de paires R2-R3 différentes: 204
  Paires R2-R3: ['R210-R311', 'R210-R3121', 'R210-R3155', 'R210-R320', 'R210-R317', 'R227-R32000', 'R227-R321', 'R227-R322', 'R227-R327', 'R227-R328', 'R227-R328', 'R27-R311', 'R27-R38', 'R271-R311', 'R271-R327', 'R271-R364', 'R271-R38', 'R271-R38']

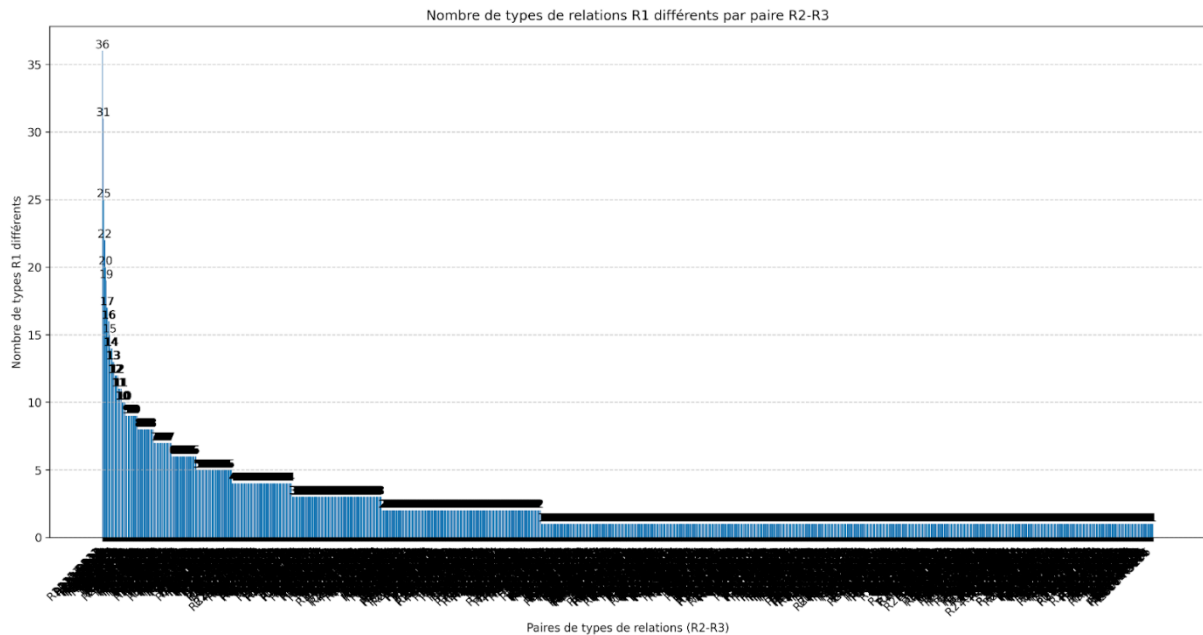
R13:
  Nombre de paires R2-R3 différentes: 196
  Paires R2-R3: ['R210-R311', 'R210-R3115', 'R210-R33', 'R210-R332', 'R210-R332', 'R23-R323', 'R23-R324', 'R23-R325', 'R23-R326', 'R23-R328', 'R23-R33', 'R23-R33', 'R25-R35', 'R25-R36', 'R25-R38', 'R254-R35', 'R254-R38', 'R254-R39', 'R26-R3']
  
```

Tous les graphiques sont générés avec un fichier txt qui détaille les résultats pour plus de lisibilité.

Cette visualisation est particulièrement utile pour :

- Identifier des **motifs structurels récurrents** dans le graphe lexical ;
- Comprendre **comment un type de relation principal est souvent associé à d'autres types de liens sémantiques** ;
- Repérer des configurations potentiellement intéressantes à étudier plus en détail dans une logique d'inférence ou d'analogie.

Nombre de bases par chapeau :



Ce graphique correspond au nombre de base (R1) trouvé par type de chapeaux (R2-R3). Il nous permettra d'identifier quelles sont les relations R1 les plus propices à avoir une paire donnée.

Détails des valeurs :

r1_types_per_hat_summaryK - Notepad

File Edit Format View Help

Résumé du nombre de types R1 différents par paire R2-R3:

R2115-R332:
Nombre de types R1 différents: 36
Types R1: [3, 5, 6, 7, 8, 9, 10, 11, 14, 15, 17, 19, 21, 22, 24, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100]

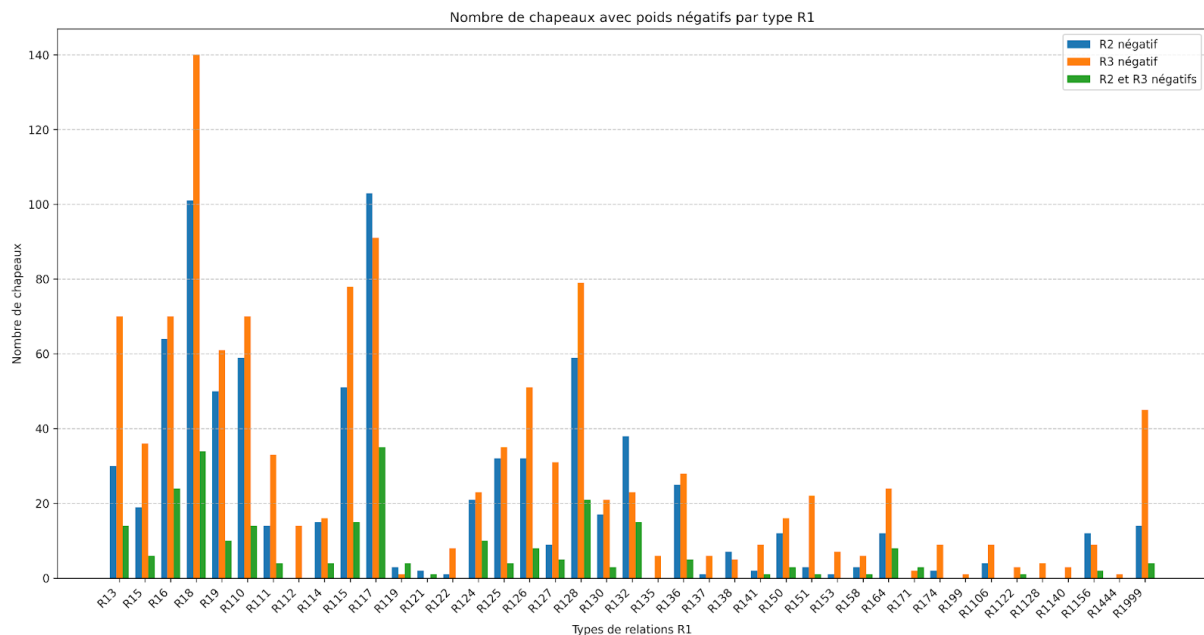
R227-R33:
Nombre de types R1 différents: 31
Types R1: [3, 5, 6, 8, 9, 10, 11, 14, 15, 17, 19, 21, 22, 23, 24, 26, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100]

R223-R317:
Nombre de types R1 différents: 25
Types R1: [3, 5, 6, 7, 8, 9, 10, 11, 15, 17, 19, 21, 23, 28, 30, 32, 35, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100]

R2115-R35:
Nombre de types R1 différents: 22
Types R1: [3, 5, 6, 8, 9, 11, 15, 17, 19, 24, 26, 27, 32, 35, 41, 56, 75, 100]

R228-R315:
Nombre de types R1 différents: 20
Types R1: [3, 5, 6, 8, 9, 10, 11, 15, 17, 19, 21, 22, 23, 28, 35, 41, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100]

Nombre de chapeau à poids négatif par base

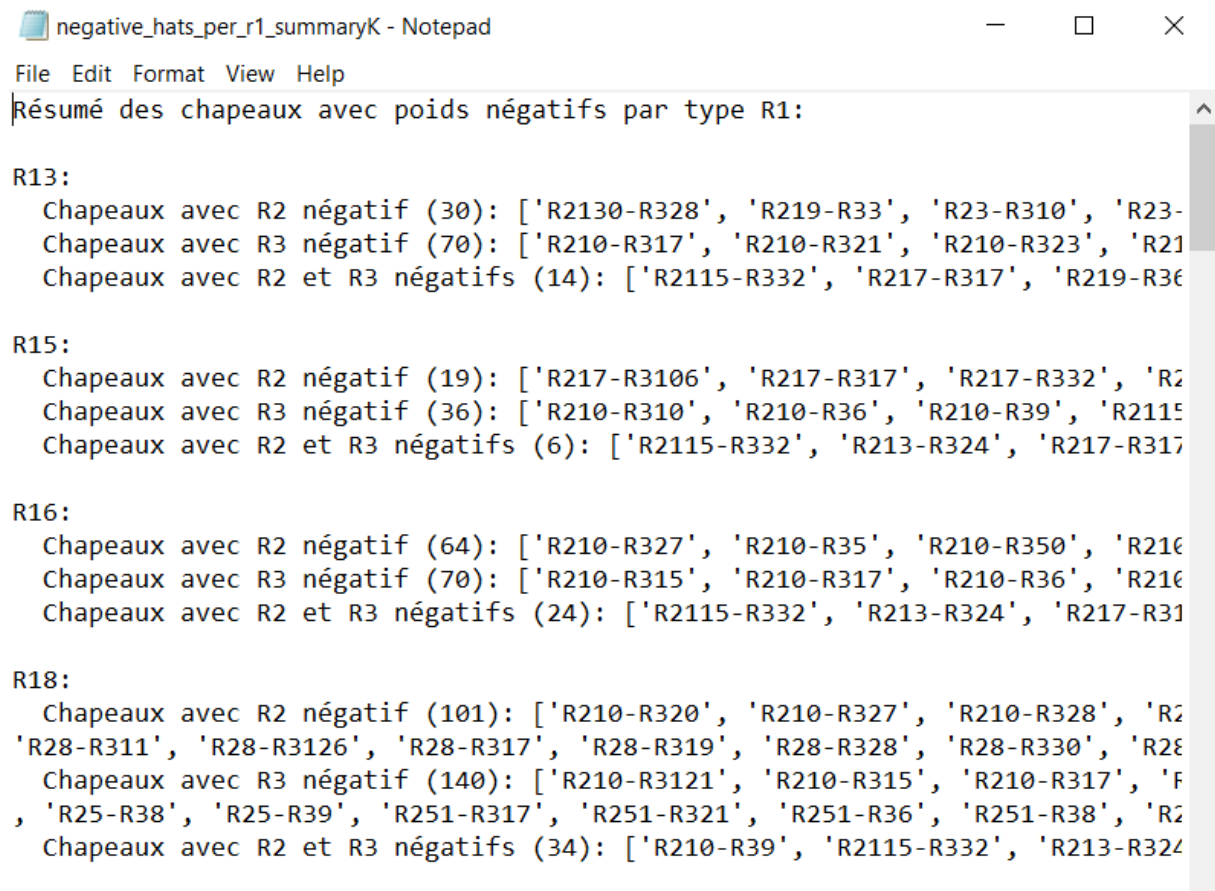


Ce graphique correspond au nombre de chapeaux (R2-R3) à poids négatifs trouvés par type de base (R1).

Il nous permettra d'identifier les relations de la base du triangle (R1) qui sont les plus propice à avoir plus de chapeau du triangle (R2-R3) à poids négatif et donc on identifie les bases les moins pertinentes.

Ce graphique est très intéressant car il permet de **visualiser quelles bases relationnelles (R1)** sont les plus susceptibles de produire des **chapeaux (R2-R3) à poids négatif**, c'est-à-dire des triangles contenant au moins une relation évaluée négativement dans JeuxDeMots.

Détails des valeurs :



```
negative_hats_per_r1_summaryK - Notepad
File Edit Format View Help
Résumé des chapeaux avec poids négatifs par type R1:

R13:
  Chapeaux avec R2 négatif (30): ['R2130-R328', 'R219-R33', 'R23-R310', 'R23-
  Chapeaux avec R3 négatif (70): ['R210-R317', 'R210-R321', 'R210-R323', 'R21
  Chapeaux avec R2 et R3 négatifs (14): ['R2115-R332', 'R217-R317', 'R219-R36

R15:
  Chapeaux avec R2 négatif (19): ['R217-R3106', 'R217-R317', 'R217-R332', 'R2
  Chapeaux avec R3 négatif (36): ['R210-R310', 'R210-R36', 'R210-R39', 'R2115
  Chapeaux avec R2 et R3 négatifs (6): ['R2115-R332', 'R213-R324', 'R217-R317

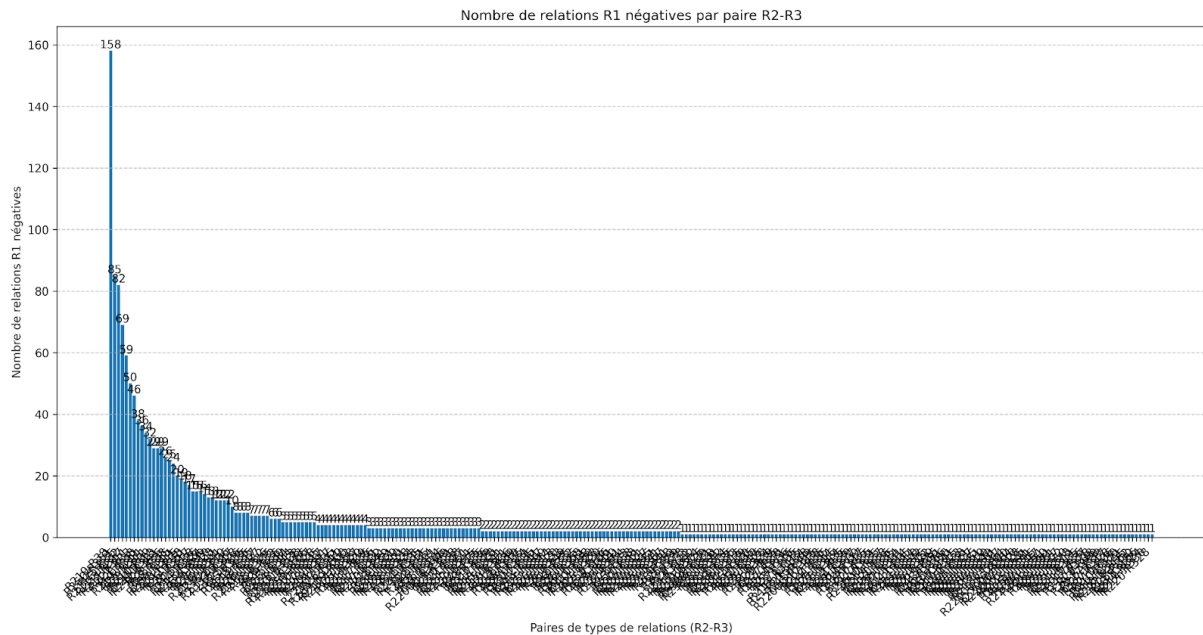
R16:
  Chapeaux avec R2 négatif (64): ['R210-R327', 'R210-R35', 'R210-R350', 'R210
  Chapeaux avec R3 négatif (70): ['R210-R315', 'R210-R317', 'R210-R36', 'R210
  Chapeaux avec R2 et R3 négatifs (24): ['R2115-R332', 'R213-R324', 'R217-R31

R18:
  Chapeaux avec R2 négatif (101): ['R210-R320', 'R210-R327', 'R210-R328', 'R2
  'R28-R311', 'R28-R3126', 'R28-R317', 'R28-R319', 'R28-R328', 'R28-R330', 'R28
  Chapeaux avec R3 négatif (140): ['R210-R3121', 'R210-R315', 'R210-R317', 'R
  , 'R25-R38', 'R25-R39', 'R251-R317', 'R251-R321', 'R251-R36', 'R251-R38', 'R2
  Chapeaux avec R2 et R3 négatifs (34): ['R210-R39', 'R2115-R332', 'R213-R324
```

En analysant ce type de distribution, il devient possible d'**identifier des zones problématiques** du graphe, ou du moins des configurations à surveiller, notamment dans une optique de nettoyage ou d'optimisation du système d'inférence.

À terme, cela pourrait conduire à **l'exclusion automatique de certains chapeaux récurrents et négatifs**, dans le but d'améliorer la qualité globale des triangles retenus.

Nombre de base à poids négatif par chapeau



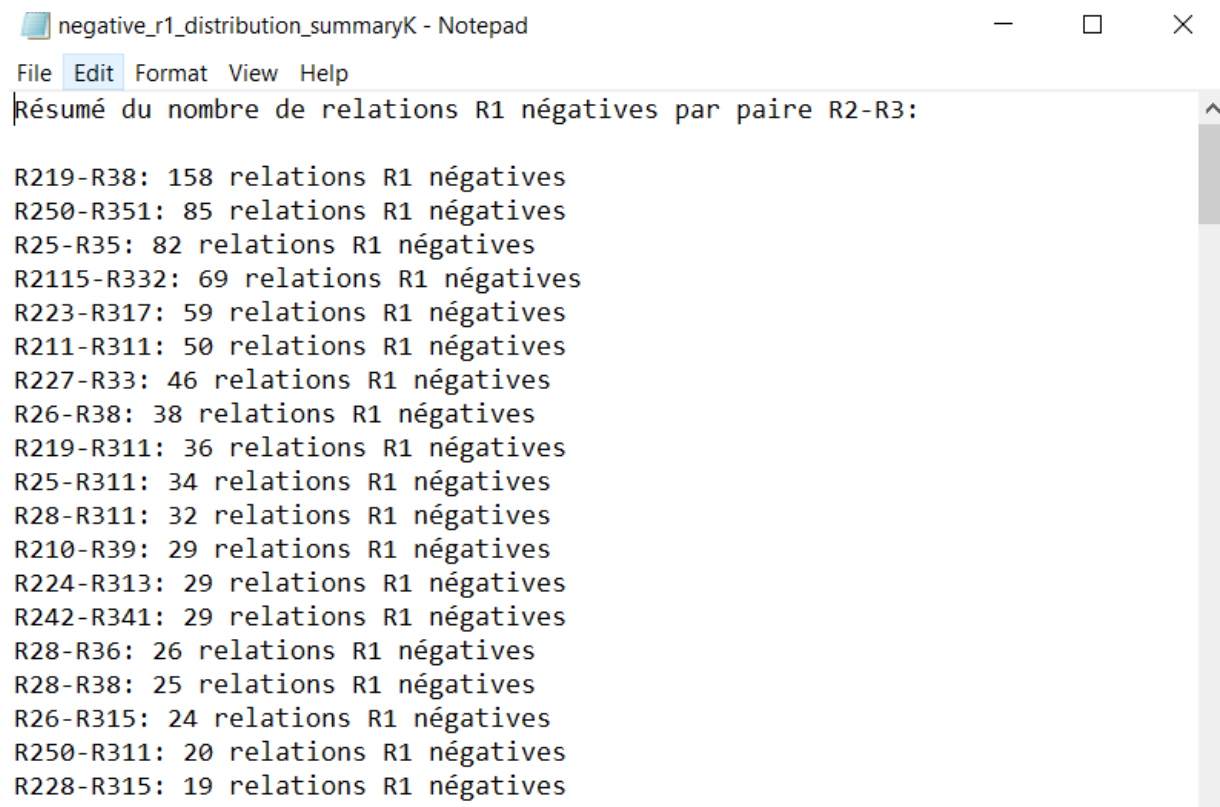
Ce graphique correspond au nombre de base (R1) à poids négatif trouvés par type de chapeaux (R2-R3).

Il nous permettra d'identifier les relations du chapeau du triangle (R2-R3) qui sont les plus propices à avoir plus de base du triangle (R1) à poids négatif et donc on identifie les chapeaux les moins pertinents.

Cela peut refléter plusieurs phénomènes :

- Une **association indirecte incohérente** : les relations R2 et R3 réunissent A et B à travers un concept C, mais cette mise en relation n'est pas jugée pertinente dans la base JeuxDeMots.
- Un **chapeau sémantiquement fragile**, c'est-à-dire un motif relationnel qui produit des triangles artificiels, ou peu convaincants du point de vue cognitif.
- Une **surinterprétation relationnelle**, où A et C ou C et B sont liés, mais le lien direct entre A et B est en réalité discutable, voire contradictoire.

Détails des valeurs :



```
negative_r1_distribution_summaryK - Notepad
File Edit Format View Help
Résumé du nombre de relations R1 négatives par paire R2-R3:

R219-R38: 158 relations R1 négatives
R250-R351: 85 relations R1 négatives
R25-R35: 82 relations R1 négatives
R2115-R332: 69 relations R1 négatives
R223-R317: 59 relations R1 négatives
R211-R311: 50 relations R1 négatives
R227-R33: 46 relations R1 négatives
R26-R38: 38 relations R1 négatives
R219-R311: 36 relations R1 négatives
R25-R311: 34 relations R1 négatives
R28-R311: 32 relations R1 négatives
R210-R39: 29 relations R1 négatives
R224-R313: 29 relations R1 négatives
R242-R341: 29 relations R1 négatives
R28-R36: 26 relations R1 négatives
R28-R38: 25 relations R1 négatives
R26-R315: 24 relations R1 négatives
R250-R311: 20 relations R1 négatives
R228-R315: 19 relations R1 négatives
-----
```

Ce type d'analyse ouvre la voie à une **approche qualitative de l'erreur dans le graphe** : en repérant les chapeaux et bases problématiques de façon systématique, il devient possible de **les filtrer ou de les marquer** comme moins fiables pour les traitements futurs.

Ces données peuvent être utiles dans des démarches de fouille de graphes sémantiques, de suggestion de relations ou encore pour enrichir automatiquement une base de connaissances en s'appuyant sur des motifs observés.

Bien que le script nous fournisse plus de graphes, ce sont principalement ces trois là qui vont le plus nous intéresser.

3. Interprétation sémantique

3.1. Explications générales

Au-delà des aspects quantitatifs, les triangles détectés peuvent être interprétés comme des structures sémantiques significatives. Chaque triangle relie trois concepts par des relations issues de la base JeuxDeMots. Ces triplets peuvent représenter :

- des chaînes d'inférence linguistique (ex. : "chat" → "poil" → "animal") ;

- des structures analogiques ou déductives ;
- ou des groupements thématiques, suggérant une proximité cognitive entre les concepts.

Toutefois, la qualité de l'interprétation dépend fortement du type de relations impliquées. Certaines relations sont plus pertinentes sémantiquement (comme "est un", "partie de", "lié à") tandis que d'autres sont moins informatives ou contextuellement floues.

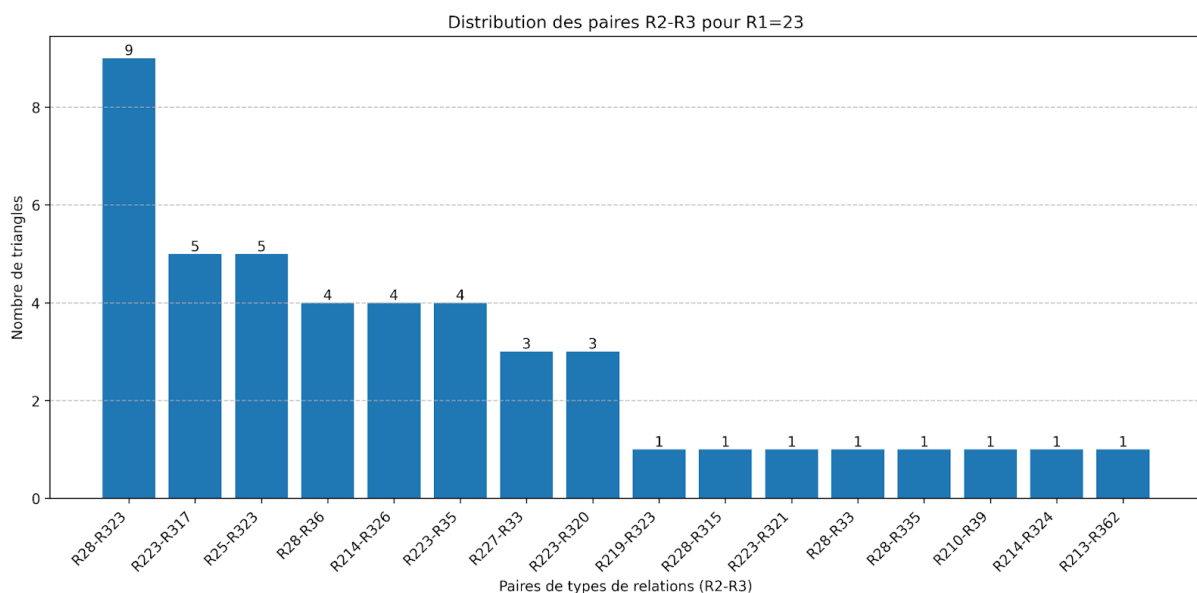
Le poids des relations joue également un rôle crucial : des poids faibles peuvent indiquer des associations plus faibles, moins fiables pour l'analyse.

Afin d'obtenir une vision globale et synthétique de la structure sémantique de notre réseau, nous avons regroupé l'ensemble des triangles extraits en un seul jeu de données, puis réalisé une série d'analyses statistiques. Ce traitement automatisé nous a permis d'identifier les motifs relationnels les plus remarquables, tant du point de vue de la diversité des relations que de la présence de poids négatifs ('w' indicateurs de tension ou d'ambiguïté sémantique).

Plus précisément, nous avons cherché à caractériser :

1. Les types de relations R1 qui génèrent le plus de diversité de chapeaux (paires R2-R3), c'est-à-dire ceux qui relient le plus grand nombre de motifs différents dans le réseau, ce qui en fait un pivot majeur de la diversité sémantique.

Voici un exemple de graphique généré sur demande de l'utilisateur par notre script qui montre la distribution des chapeaux pour une base de triangles (R1=23) :



2. Les types R1 qui, au contraire, sont associés au moins de chapeaux négatifs, c'est-à-dire ceux pour lesquels très peu de paires R2-R3 présentent un poids négatif, ce qui peut indiquer une plus grande stabilité ou cohérence sémantique pour ces relations.

Nous avons également analysé les paires de types de relations secondaires (R2-R3) :

1. Celles qui sont présentes dans le plus grand nombre de types R1 différents, ce qui révèle des motifs structurels très transversaux et récurrents dans le réseau.
2. Celles qui, à l'inverse, sont associées au moins de R1 négatifs, ce qui peut signaler des motifs particulièrement robustes ou consensuels.

Enfin, nous avons extrait les paires R2-R3 les plus « robustes » (celles qui cumulent le moins de R1 négatifs tout en étant présentes dans de nombreux types R1 différents).

Cette double approche permet de repérer à la fois les points d'ancrage stables du réseau sémantique et les zones de tension ou d'ambiguïté, offrant ainsi une cartographie fine des dynamiques relationnelles à l'œuvre dans notre corpus.

Tous ces résultats ont été obtenus grâce à un code Python qui a permis d'automatiser le calcul et le tri de ces motifs, rendant possible une analyse à grande échelle et reproductible sur l'ensemble des données.

Les résultats que nous allons voir aujourd'hui se basent sur 7 mots ayant maximum 10 000 triangles chacun.

3.2. Bases pertinentes

Parmi les types de relations analysés en profondeur via les paires R2–R3 (les "chapeaux"), certains se distinguent également par leur **diversité structurelle** : c'est-à-dire qu'ils donnent lieu à un **grand nombre de combinaisons différentes de relations secondaires** dans les triangles.

On observe notamment :

- **Type 128** avec **681 paires R2–R3 distinctes** : c'est la relation la plus diversifiée, couvrant un large éventail de schémas d'inférence.
- **Type 18** avec **476 paires** : elle se positionne comme la deuxième plus riche en diversité structurelle, révélant une forte connectivité sémantique.
- **Type 15** avec **470 paires** : très proche de la précédente, elle témoigne également d'une capacité à s'insérer dans une grande variété de triangles.

Cette richesse en paires R2–R3 suggère que ces relations, bien qu'éventuellement moins fréquentes en volume global que d'autres types, **jouent un rôle fondamental dans la diversité du graphe**. Elles permettent de générer **de nombreux chemins d'inférence différents**, ce qui les rend précieuses dans des contextes d'analyse fine ou de génération d'hypothèses linguistiques.

3.3. Chapeaux Pertinents

Nous voici maintenant à l'une des parties les plus intéressantes de ce projet : **quels sont les « chapeaux » les plus pertinents ?**

Par « chapeau », nous désignons ici la **combinaison des deux relations secondaires** dans un triangle, c'est-à-dire **R2 (C → B) et R3 (A → C)**.

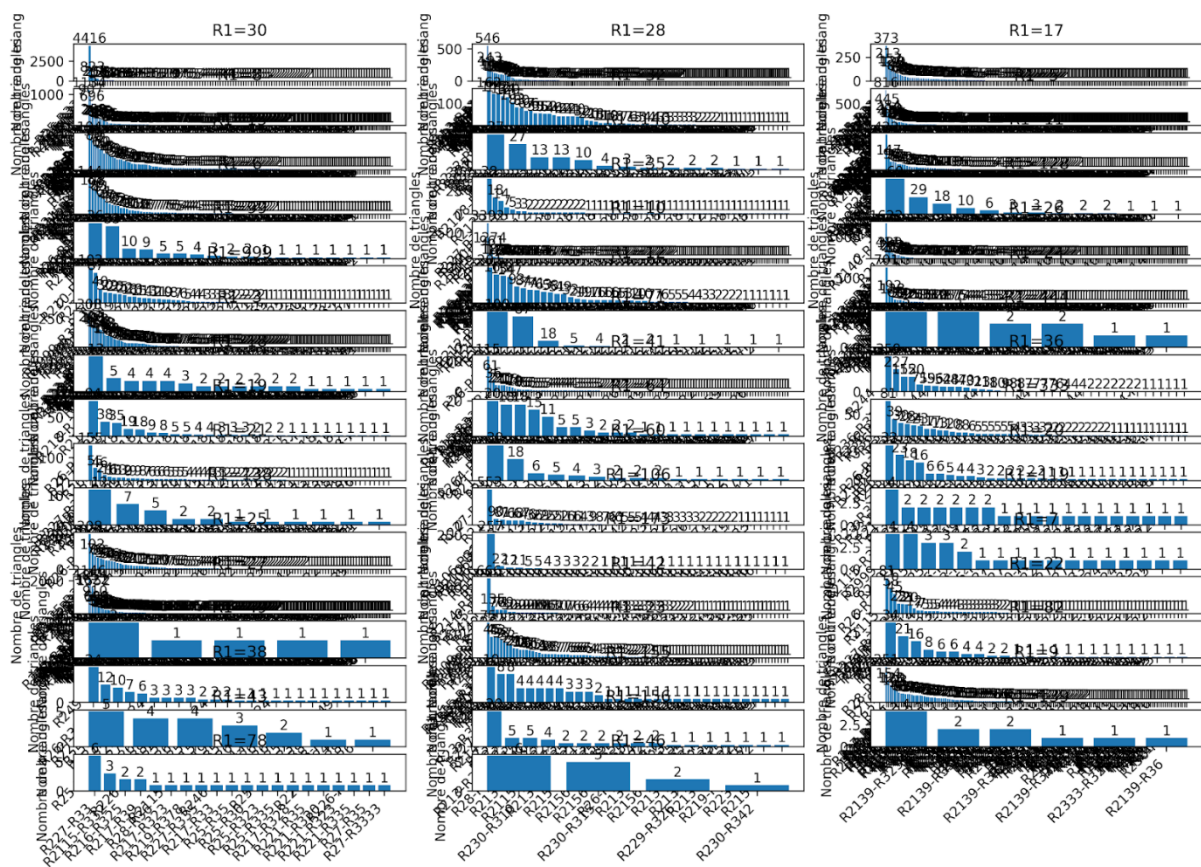
Ces paires R2–R3 peuvent être vues comme des **schémas d'inférence**, car elles définissent la manière dont un concept C permet de relier A à B de façon indirecte, en complétant la relation principale

R1 (A → B)

C'est donc à travers ces « chapeaux » que le triangle acquiert une **cohérence structurelle**, voire sémantique.

Cette analyse permet de faire émerger des **motifs récurrents** dans le graphe lexical, et d'identifier **les formes d'inférence les plus fréquentes ou les plus efficaces** selon les types de relations impliquées.

Voici le graphique de toutes les relations obtenues :



Intéressons-nous plus particulièrement à la relation de type 8 :

Résumé des paires de types de relations par R1:

Pour R1=8:

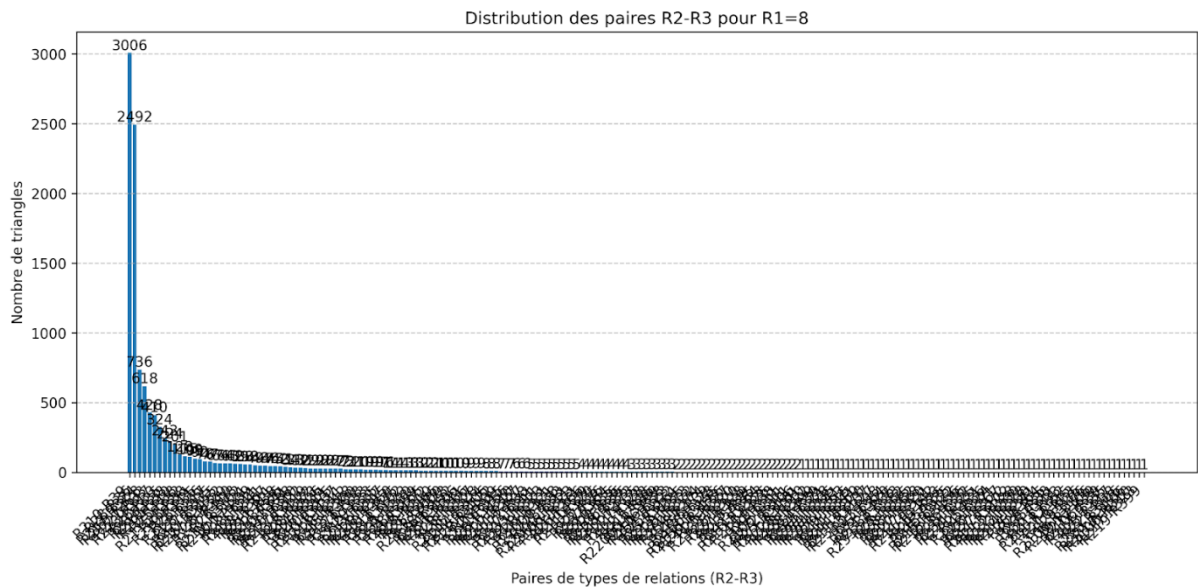
R210-R39: 1939 triangles
R28-R36: 1603 triangles
R26-R38: 1185 triangles
R223-R317: 989 triangles
R227-R33: 986 triangles
R219-R38: 855 triangles
R213-R324: 809 triangles
R2115-R332: 606 triangles
R28-R38: 563 triangles
R25-R38: 558 triangles

On peut voir ici que **deux chapeaux se distinguent nettement des autres**. Le premier est **R2 10–R3 9** (respectivement : r_holo et r_has_part) , présent dans **1939 triangles**, et le second est **R2 8–R3 6** (respectivement : r_hypo et r_isa), qui apparaît **2492 fois**. À eux deux, ces schémas couvrent à eux seuls **près de 49 %** des triangles détectés pour **R1 = 8**, ce qui en fait des **motifs structurants majeurs** dans le graphe.

Cela signifie que, lorsqu'une relation principale de type 8 relie deux concepts ($A \rightarrow B$), les structures triangulaires les plus fréquentes impliquent :

- soit une relation **R210 de C vers B** et **R39 de A vers C** ;
- soit une relation **R28 de C vers B** et **R36 de A vers C**.

Nous pouvons comparer ces résultats issus de notre test final à ceux d'un test fait précédemment qui comptait seulement les relations positives avec un total de **74 443 triangles**.



1	Résumé des paires de types de relations pour R1=8:
2	
3	Nombre total de triangles: 11207
4	
5	R210-R39: 3006 triangles
6	R28-R36: 2492 triangles
7	R213-R324: 736 triangles
8	R210-R328: 618 triangles
9	R227-R33: 428 triangles
10	R223-R317: 410 triangles
11	R28-R33: 324 triangles
12	R28-R335: 243 triangles
13	R219-R38: 224 triangles
14	R29-R310: 201 triangles
15	R25-R38: 128 triangles

Ici, les résultats sont très similaires aux premiers que nous avons eu. Avec les deux premières paires qui couvrent à elles seules **près de 49 %** des triangles détectés pour **R1 = 8**, ce qui en fait des **motifs structurants majeurs** dans le graphe.

Cela confirme la **stabilité et la robustesse de ces schémas d'inférence**, qui apparaissent comme des **structures sémantiques fiables**, quel que soit le jeu de données ou le filtrage appliqué.

Nous allons aussi nous intéresser à la base (relation $A \rightarrow B$) de type 28 :

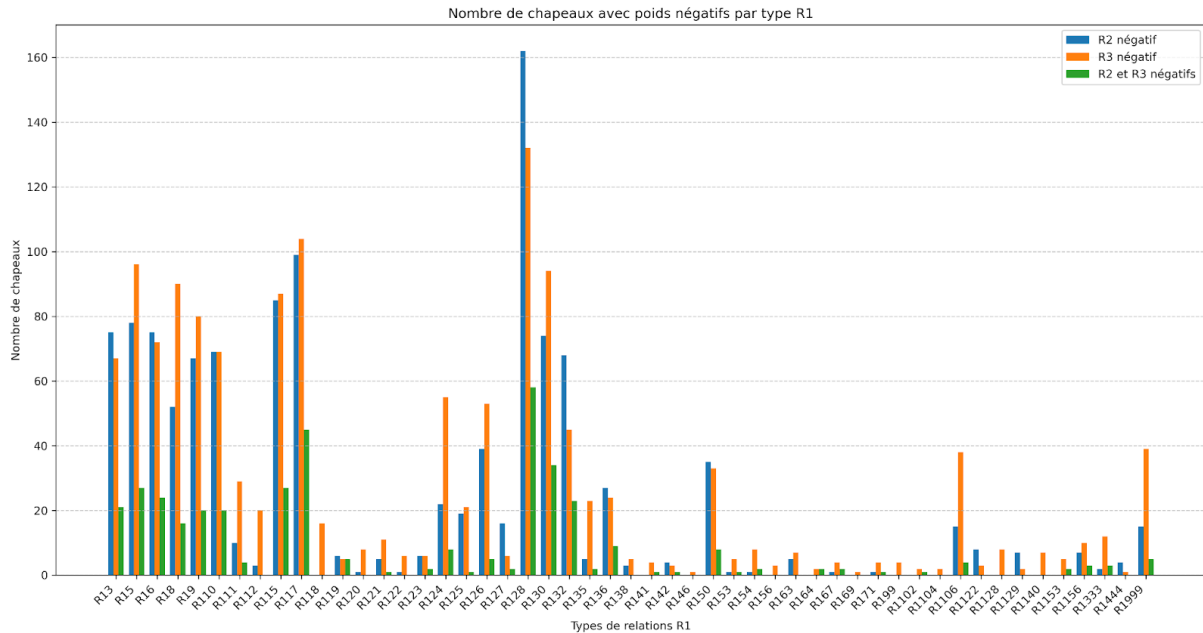
Pour $R1=28$:

R228-R315: 2557 triangles
R223-R317: 2487 triangles
R2115-R332: 2144 triangles
R213-R330: 1831 triangles
R228-R310: 1586 triangles
R216-R330: 1360 triangles
R28-R328: 1310 triangles
R227-R33: 1293 triangles
R25-R328: 1148 triangles
R214-R330: 972 triangles

Ici aussi, le type de la base apparaît parmi les premières relations. Comme pour le type 8 il apparaît également pour R2, il est donc approprié de se demander si **certaines relations tendent naturellement à se répéter dans différentes positions du triangle**, ce qui pourrait indiquer **des schémas relationnels récurrents et structurants dans le graphe**.

3.4. Chapeaux non-pertinents

Pour cette partie nous allons analyser 2 graphiques qui vont nous permettre de tout comprendre.

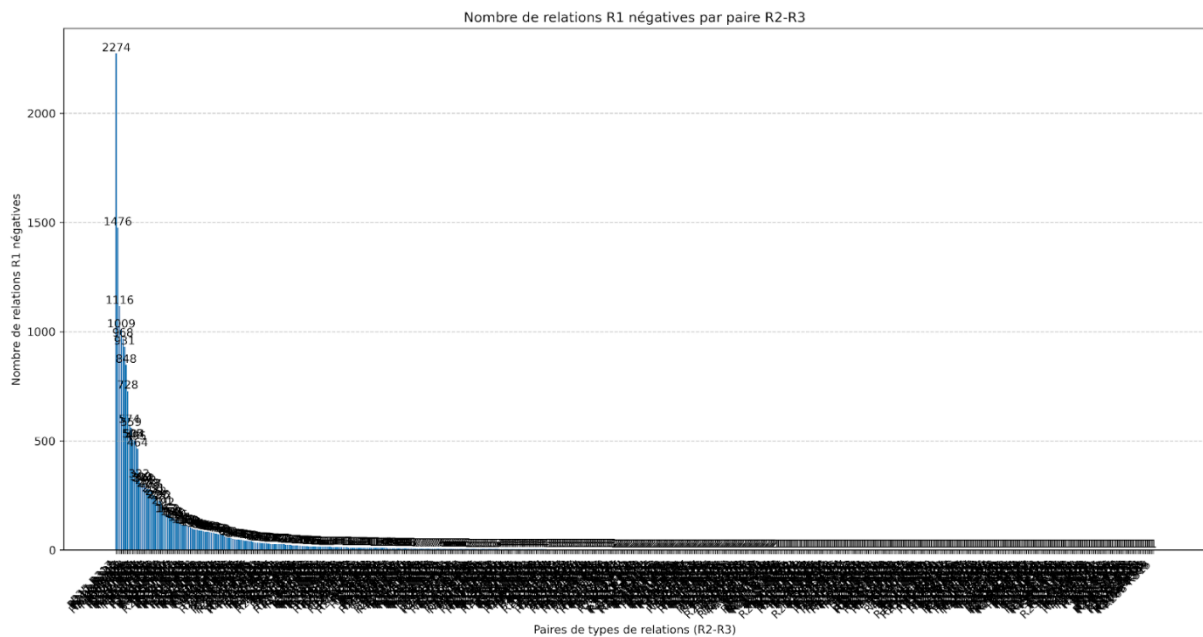


Ici, nous voyons les bases les plus susceptibles de générer des triangles avec des relations négatives. Une nous saute aux yeux directement et c'est la relation de type 28 avec des relations négatives dépassant les 160 pour R2. On peut donc se demander si le nombre de chapeaux assez important que nous avons eu est assez pertinent. Et si les relations sont elles aussi pertinentes car ce graphique laisse à penser qu'il y a quelques soucis avec la relation de type 28.

Nous pouvons également regarder la relation de type 8 étant donnée que nous l'avons considérée comme très pertinente. Elle a elle aussi quelques soucis avec à peu près 90 relations négatives pour R3 mais cela reste malgré tout léger en comparaison avec les résultats juste qu'elle nous fournit.

Dans ce graphique il est important de noter que les relations $A \rightarrow C$ (R3) ont plus de chances d'être négatives.

Nous pouvons maintenant passer au graphique suivant :



Il est assez peu compréhensible mais on peut voir une grosse distinction entre la première paire et la deuxième c'est pourquoi nous allons nous concentrer sur celle-là :

Résumé du nombre de relations R1 négatives par paire R2-R3:

R25-R317: 2274 relations R1 négatives
R217-R328: 1476 relations R1 négatives
R213-R330: 1116 relations R1 négatives
R228-R315: 1009 relations R1 négatives
R223-R317: 968 relations R1 négatives
R2115-R332: 931 relations R1 négatives
R217-R315: 848 relations R1 négatives
R28-R36: 728 relations R1 négatives
R230-R315: 574 relations R1 négatives

Les relations $C \rightarrow B$ de type 5 et $A \rightarrow C$ de type 17 sont plus susceptibles d'avoir des relations $A \rightarrow B$ négatives.

Ces chiffres indiquent que **ces chapeaux particuliers** apparaissent souvent dans des triangles problématiques du point de vue de la relation $A \rightarrow B$. Ils pourraient ainsi être considérés comme **des configurations moins fiables**, ou du moins comme des schémas d'inférence **statistiquement associés à des résultats négatifs**.

Ce type de données est précieux pour envisager des stratégies de **filtrage automatique**, en identifiant les paires R2-R3 les plus susceptibles de conduire à des liens faibles ou indésirables dans le graphe.

3. Discussion sur les résultats

4.1. Discussion générale

Les résultats obtenus à l'issue de ce projet révèlent une **structuration non aléatoire du graphe sémantique**. La quantité importante de triangles détectés, leur diversité, ainsi que la fréquence de certaines combinaisons relationnelles indiquent que **des motifs récurrents** émergent naturellement au sein du réseau lexical. Ces motifs peuvent être interprétés comme des **schémas d'inférence linguistique**, ce qui ouvre la voie à des usages plus avancés, comme l'enrichissement automatique de graphes ou la suggestion de relations implicites.

La concentration de triangles autour de certains types de relations (ex. : $R1 = 8, 6, 5, 24$) montre que certaines catégories de liens jouent un **rôle central dans la structuration du lexique**. À l'inverse, d'autres relations (comme $R1 = 71$ ou $R1 = 99$) se sont révélées marginales, voire anecdotiques, suggérant qu'elles sont moins utiles dans une logique d'inférence ou de structuration sémantique.

L'approche par visualisation a également permis de mieux comprendre les dynamiques internes du graphe : les "chapeaux" (paires $R2-R3$) récurrents montrent des **chemins cognitifs ou linguistiques préférentiels**, ce qui donne du sens aux triangles les plus fréquents. Ces éléments pourront servir de base pour des recherches futures sur la générativité sémantique, ou la modélisation de raisonnements lexicaux.

Enfin, bien que certaines limites soient encore présentes (notamment en ce qui concerne le filtrage des nœuds et des relations), le projet a démontré que la **détection de triangles est une méthode simple mais puissante** pour explorer et analyser la structure profonde d'un graphe lexical comme celui de JeuxDeMots.

4.2. Améliorations

Bien que nous commencions à distinguer certaines tendances intéressantes dans les triangles détectés, il est important de souligner que des **améliorations restent nécessaires**. La plus significative, selon nous, serait de **basculer le calcul des statistiques directement sur la base de données**.

Comme mentionné précédemment, la base de données SQLite que nous avons intégrée est pour l'instant utilisée uniquement comme **espace de stockage passif**. Cependant, à terme, il serait pertinent de **centraliser l'ensemble du traitement statistique autour de cette base**.

Cela permettrait non seulement d'automatiser des analyses plus complexes (agrégations, requêtes spécifiques, filtrage par type ou poids de relation), mais aussi de gagner en performance et en lisibilité.

En effet, exécuter le script complet sur un très grand volume de données est coûteux en temps et en ressources. **Externaliser une partie de l'analyse vers la base de données** permettrait de **travailler plus efficacement sur des sous-ensembles ciblés**, sans devoir relancer le traitement entier à chaque modification de paramètre.

Cette évolution irait dans le sens d'un projet plus modulaire, plus robuste, et plus exploitable à long terme.

Par ailleurs, il serait essentiel de **renforcer le filtrage des relations et des nœuds** utilisés pour construire les triangles. Actuellement, certains nœuds sont problématiques : on retrouve notamment des termes en anglais, ou pire, des nœuds incompréhensibles, composés uniquement de chaînes alphanumériques sans signification claire. Ces éléments nuisent à la qualité sémantique des triangles formés et peuvent **brouiller l'interprétation des résultats**.

Le même constat s'applique aux **types de relations**. Certaines, trop génériques ou peu informatives, apportent peu de valeur ajoutée à l'analyse. En les excluant, nous pourrions non seulement **affiner la qualité des triangles détectés**, mais aussi **réduire le temps de traitement** et la complexité des visualisations générées.

Aussi, il serait intéressant de chercher à optimiser au maximum le script afin de rendre son exécution plus rapide. Notre code doit s'occuper de tellement de choses qu'il finit par être un peu lent ce qui nous freine dans nos analyses.

Enfin, il serait judicieux d'**élargir les tests à des ensembles de mots plus variés**, afin de mieux évaluer la robustesse et la pertinence des triangles générés. Jusqu'à présent, une grande partie des tests a été réalisée à partir de **noms communs et de noms propres**, ce qui a pu influencer les structures relationnelles rencontrées.

Pour approfondir l'analyse, il serait intéressant de **répéter l'expérience avec des verbes, des adjectifs ou des expressions complexes**. Cela permettrait d'explorer d'autres dynamiques sémantiques, de vérifier si les schémas d'inférence sont aussi fréquents et cohérents dans d'autres catégories grammaticales, et d'éventuellement **identifier des motifs spécifiques à certaines natures de mots**.

CONCLUSION

L'exploration des triangles sémantiques au sein du réseau lexical JeuxDeMots a permis de franchir une étape significative dans la compréhension des dynamiques internes qui structurent le lexique français. Ce travail s'inscrit dans une démarche à la fois descriptive, analytique et prospective, visant à révéler les mécanismes sous-jacents à l'organisation sémantique des concepts et à la circulation du sens dans un vaste graphe lexical.

L'analyse systématique des triangles, c'est-à-dire des triplets de relations reliant trois concepts via une relation principale (R1) et deux relations secondaires (R2, R3), a mis en évidence l'existence de motifs récurrents, appelés « chapeaux ». Ces chapeaux, loin d'être de simples artefacts statistiques, traduisent des schémas d'inférence privilégiés qui témoignent d'une structuration profonde du lexique. Ils révèlent comment certains types de relations secondaires, en se combinant, permettent d'établir des liens indirects mais sémantiquement cohérents entre des concepts parfois éloignés. Cette propriété confère au réseau une grande souplesse et une capacité d'expansion sémantique, essentielle pour la compréhension et la génération de sens.

Les résultats statistiques obtenus sont particulièrement révélateurs : le nombre élevé de triangles détectés, la diversité des types de relations impliquées, ainsi que la fréquence de certains chapeaux, confirment l'existence d'une organisation hiérarchique et modulaire du réseau. Les relations fondamentales, identifiées par leur omniprésence et leur diversité, constituent le socle du graphe, tandis que les relations spécialisées et négatives apportent des nuances, des oppositions et une granularité fine à l'ensemble. L'apparition de clusters sémantiques, de motifs d'inférence récurrents et de zones de forte connectivité suggère que le lexique fonctionne comme un système dynamique, optimisé pour la navigation, l'apprentissage et la robustesse face à l'ambiguïté.

D'un point de vue méthodologique, ce travail a permis de valider l'intérêt d'une approche fondée sur l'extraction et l'analyse de motifs structuraux dans les graphes lexicaux. La méthodologie développée, alliant extraction algorithmique, visualisation et interprétation sémantique, s'est révélée efficace pour mettre au jour des propriétés globales et locales du réseau. Elle ouvre la voie à des analyses comparatives entre langues, à l'étude de l'évolution diachronique des réseaux lexicaux, ou encore à l'exploration de sous-graphes thématiques.

Les perspectives applicatives sont nombreuses. Dans le domaine du traitement automatique du langage naturel, la prise en compte des triangles sémantiques et des chapeaux pourrait enrichir les modèles de représentation du sens, améliorer les systèmes de désambiguïsation lexicale, ou encore guider la génération automatique de texte en s'appuyant sur des schémas d'inférence validés empiriquement. Plus largement, cette approche pourrait être transposée à

d'autres types de réseaux de connaissances, qu'il s'agisse de bases terminologiques, d'ontologies ou de graphes de connaissances utilisés en intelligence artificielle.

Enfin, ce travail met en lumière l'importance de considérer non seulement les relations directes entre concepts, mais aussi les chemins d'inférence indirects qui participent à la richesse, à la cohérence et à la flexibilité du lexique. L'étude des triangles sémantiques apparaît ainsi comme une clé de lecture privilégiée pour comprendre la dynamique du sens dans les réseaux lexicaux, et ouvre des perspectives prometteuses pour la modélisation, l'analyse et l'exploitation des ressources linguistiques à grande échelle.

BIBLIOGRAPHIE

Definition Endpoint : <https://www.yieldstudio.fr/glossaire/endpoint>

Rezo JeuxDeMots : <https://www.jeuxdemots.org/rezo.php>

Relations définition : <https://www.jeuxdemots.org/jdm-about-detail-relations.php>

Définition TAL : https://fr.wikipedia.org/wiki/Traitement_automatique_des_langues

API JeuxDeMots : <https://jdm-api.demo.lirmm.fr/schema#operation/RedirectSchema>

Définition API : [https://www.cnil.fr/fr/definition/interface-de-programmation-dapplication-api#:~:text=Une%20API%20\(application%20programming%20interface,des%20donn%C3%A9es%20et%20des%20fonctionnalit%C3%A9s](https://www.cnil.fr/fr/definition/interface-de-programmation-dapplication-api#:~:text=Une%20API%20(application%20programming%20interface,des%20donn%C3%A9es%20et%20des%20fonctionnalit%C3%A9s)

ETAT DE L'ART :

<https://medium.com/data-science/a-decade-of-knowledge-graphs-in-natural-language-processing-5fdb15abc2b3>

https://www.researchgate.net/publication/221012612_Online_Word_Games_for_Semantic_Data_Collection

https://www.researchgate.net/publication/373931770_Link_Prediction_Based_on_the_Relational_Path_Inference_of_Triangular_Structures

https://people.csail.mit.edu/virgi/6.890/lecture10.pdf?utm_source=chatgpt.com